

# 100 Python Questions & Answers

---

## Table of Contents

---

1. [Python Basics](#)
  2. [Object-Oriented Programming](#)
  3. [NumPy](#)
  4. [Pandas](#)
  5. [Matplotlib](#)
- 

## 1. Python Basics (25 Questions)

---

### Question 1: Conceptual

What are the key differences between a list and a tuple in Python?

**Answer:** Lists and tuples are both used to store collections of items in Python, but they differ primarily in their mutability and syntax.

- **Mutability:** Lists are **mutable**, meaning their elements can be changed (added, removed, or modified) after creation. Tuples are **immutable**, meaning their elements cannot be changed once the tuple is created.
- **Syntax:** Lists are defined using square brackets `[]`, while tuples are defined using parentheses `()`. Tuples can also be defined without parentheses, simply by separating items with commas.

### Question 2: Coding

Write a Python function `reverse_string(s)` that takes a string `s` as input and returns the string reversed.

**Answer:**

```
def reverse_string(s):  
    return s[::-1]  
  
# Test cases  
print(reverse_string("hello")) # Output: olleh  
print(reverse_string("Python")) # Output: nohtyP
```

**Explanation:** The `[::-1]` slicing technique creates a reversed copy of the string. The first colon indicates the start of the slice, the second colon indicates the end, and `-1` specifies a step of -1, which reverses the sequence.

### Question 3: Output Prediction

What will be the output of the following Python code?

```
x = [1, 2, 3]  
y = x  
y.append(4)  
print(x)
```

**Answer:**

```
[1, 2, 3, 4]
```

**Explanation:** In Python, when you assign a list to another variable (e.g., `y = x`), you are not creating a copy of the list. Instead, both variables `x` and `y` refer to the *same* list object in memory. Therefore, when `y.append(4)` modifies the list, the change is reflected in `x` as well.

### Question 4: Debugging/Correction

The following code is intended to calculate the sum of all even numbers in a list. Identify and correct the error.

```
def sum_even_numbers(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 1: # Incorrect condition
            total += num
    return total

my_list = [1, 2, 3, 4, 5, 6]
print(sum_even_numbers(my_list))
```

### Corrected Code:

```
def sum_even_numbers(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 0: # Corrected condition: check for even numbers
            total += num
    return total

my_list = [1, 2, 3, 4, 5, 6]
print(sum_even_numbers(my_list)) # Output: 12 (2 + 4 + 6)
```

**Explanation:** The original code had an incorrect condition `if num % 2 == 1:`, which checks if a number is odd. To sum even numbers, the condition should be `if num % 2 == 0:`, which correctly identifies even numbers.

### Question 5: Conceptual

Explain the concept of **scope** in Python and differentiate between local and global scope.

**Answer: Scope** in Python refers to the region of a program where a variable is accessible. It determines the visibility of a variable within different parts of your code.

- **Local Scope:** A variable defined inside a function has a **local scope**. It is only accessible from within that function. Once the function finishes execution, the local variables are destroyed.

- **Global Scope:** A variable defined outside of any function, at the top level of a script, has a **global scope**. It can be accessed from anywhere in the program, both inside and outside functions. However, to modify a global variable inside a function, you must explicitly declare it using the `global` keyword.

## Question 6: Coding

Write a Python function `factorial(n)` that calculates the factorial of a non-negative integer `n` using recursion.

**Answer:**

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

# Test cases
print(factorial(0)) # Output: 1
print(factorial(5)) # Output: 120 (5 * 4 * 3 * 2 * 1)
print(factorial(7)) # Output: 5040
```

**Explanation:** The `factorial` function uses recursion. The base case is when `n` is 0, returning 1. For any other `n`, it returns `n` multiplied by the factorial of `n-1`, effectively breaking down the problem into smaller, similar subproblems until the base case is reached.

## Question 7: Output Prediction

What will be the output of the following Python code?

```
def greet(name, message="Hello"):
    print(f"{message}, {name}!")

greet("Alice")
greet("Bob", "Hi")
```

## Answer:

```
Hello, Alice!  
Hi, Bob!
```

**Explanation:** The `greet` function has a default argument `message="Hello"`. When `greet("Alice")` is called, `message` defaults to “Hello”. When `greet("Bob", "Hi")` is called, “Hi” overrides the default value for `message`.

## Question 8: Debugging/Correction

The following code attempts to create a list of squares of numbers from 1 to 5 using a list comprehension, but it has a syntax error. Correct it.

```
squares = [x**2 for x in range(1, 6) if x % 2 == 0]  
print(squares)
```

### Corrected Code:

```
squares = [x**2 for x in range(1, 6)] # No syntax error in original, but if  
the intent was all numbers, remove the if condition  
print(squares) # Output: [1, 4, 9, 16, 25]
```

**Explanation:** Actually, the original code `squares = [x**2 for x in range(1, 6) if x % 2 == 0]` is syntactically correct and would produce `[4, 16]` (squares of even numbers). If the intent was to get squares of *all* numbers from 1 to 5, the `if x % 2 == 0` condition should be removed. Assuming the user meant a general list of squares, the corrected code removes the filtering condition.

## Question 9: Conceptual

What is a **lambda function** in Python, and when would you typically use it?

**Answer:** A **lambda function** (also known as an anonymous function) is a small, single-expression function that can be defined without a name. It is created using the

`lambda` keyword.

Lambda functions are typically used for:

- **Short, one-time operations:** When you need a simple function for a brief period and don't want to formally define it using `def`.
- **Higher-order functions:** As arguments to functions that take other functions as input, such as `map()`, `filter()`, and `sorted()`.

**Example:**

```
# Using lambda with sorted()
points = [(1, 2), (3, 1), (5, 0)]
sorted_points = sorted(points, key=lambda point: point[1])
print(sorted_points) # Output: [(5, 0), (3, 1), (1, 2)]
```

## Question 10: Coding

Write a Python function `count_vowels(text)` that takes a string `text` and returns the number of vowels (a, e, i, o, u, case-insensitive) in it.

**Answer:**

```
def count_vowels(text):
    vowels = "aeiou"
    count = 0
    for char in text:
        if char.lower() in vowels:
            count += 1
    return count

# Test cases
print(count_vowels("Hello World")) # Output: 3
print(count_vowels("Python Programming")) # Output: 5
```

**Explanation:** The function iterates through each character in the input `text`. It converts each character to lowercase using `char.lower()` and checks if it exists in the `vowels` string. If it's a vowel, the `count` is incremented.

## Question 11: Output Prediction

What will be the output of the following Python code?

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
my_dict['b'] = 4
my_dict['d'] = 5
print(my_dict.pop('a'))
print(my_dict)
```

**Answer:**

```
1
{'b': 4, 'c': 3, 'd': 5}
```

**Explanation:**

- `my_dict['b'] = 4` updates the value associated with key 'b'.
- `my_dict['d'] = 5` adds a new key-value pair 'd': 5.
- `my_dict.pop('a')` removes the key 'a' and its value from the dictionary and returns the value (which is 1). The `print()` statement then outputs this returned value.
- The final `print(my_dict)` shows the dictionary after these operations.

## Question 12: Debugging/Correction

The following code is supposed to merge two dictionaries. Identify and correct the error.

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged_dict = dict1.update(dict2) # Incorrect usage
print(merged_dict)
```

**Corrected Code (Method 1: Using `update()`):**

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
dict1.update(dict2) # update() modifies dict1 in-place and returns None
merged_dict = dict1 # Assign the modified dict1 to merged_dict
print(merged_dict) # Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

### Corrected Code (Method 2: Using `**` operator for Python 3.5+):

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged_dict = {**dict1, **dict2} # Creates a new dictionary by unpacking
both
print(merged_dict) # Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

**Explanation:** The `update()` method of a dictionary modifies the dictionary *in-place* and returns `None`. In the original code, `merged_dict` was assigned `None`. To correctly merge using `update()`, you should call `dict1.update(dict2)` and then `merged_dict` would refer to the modified `dict1`. A more Pythonic way for Python 3.5+ is to use the `**` operator to unpack dictionaries into a new one.

### Question 13: Conceptual

What is the purpose of the `if __name__ == "__main__":` block in Python scripts?

**Answer:** The `if __name__ == "__main__":` block is a common idiom in Python scripts that allows you to define code that should only run when the script is executed directly, not when it's imported as a module into another script.

- When a Python script is run directly, the special variable `__name__` is set to `"__main__"`.
- When the script is imported as a module, `__name__` is set to the module's name.

This block is useful for:

- **Testing:** Including test code that only runs when you're testing the script itself.
- **Initialization:** Running setup code or the main logic of your application only when it's the primary script being executed.



## Question 14: Coding

Write a Python function `find_max(numbers)` that takes a list of numbers and returns the largest number in the list without using the built-in `max()` function.

**Answer:**

```
def find_max(numbers):
    if not numbers:
        return None # Handle empty list case

    maximum = numbers[0]
    for num in numbers:
        if num > maximum:
            maximum = num
    return maximum

# Test cases
print(find_max([1, 5, 2, 8, 3])) # Output: 8
print(find_max([-10, -2, -5])) # Output: -2
print(find_max([])) # Output: None
```

**Explanation:** The function first checks if the list is empty. If not, it initializes `maximum` with the first element. Then, it iterates through the rest of the list, updating `maximum` whenever a larger number is found.

## Question 15: Output Prediction

What will be the output of the following Python code?

```
def modify_list(lst):
    lst.append(4)
    lst = [10, 20, 30]

my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)
```

**Answer:**

```
[1, 2, 3, 4]
```

**Explanation:** When `modify_list(my_list)` is called, `lst` initially refers to the same list object as `my_list`. `lst.append(4)` modifies this shared list, so `my_list` becomes `[1, 2, 3, 4]`. However, the line `lst = [10, 20, 30]` inside the function reassigns the local variable `lst` to a *new* list object. This reassignment does not affect the `my_list` variable in the global scope, which still points to the original (now modified) list.

## Question 16: Debugging/Correction

The following code attempts to read a file and print its content, but it might lead to resource leaks. Correct it to ensure the file is properly closed.

```
# file_content.txt contains: Hello World

file = open("file_content.txt", "r")
content = file.read()
print(content)
# file.close() is missing
```

### Corrected Code (Method 1: Using `try-finally`):

```
# Assume 'file_content.txt' exists with some content
try:
    file = open("file_content.txt", "r")
    content = file.read()
    print(content)
finally:
    file.close() # Ensures file is closed even if errors occur
```

### Corrected Code (Method 2: Using `with` statement - Recommended):

```
# Assume 'file_content.txt' exists with some content
with open("file_content.txt", "r") as file:
    content = file.read()
    print(content)
# File is automatically closed when exiting the 'with' block
```

**Explanation:** The original code was missing `file.close()`, which is crucial for releasing system resources associated with the file. If an error occurred before `file.close()` was called, the file would remain open. The `with` statement (context manager) is the most Pythonic and recommended way to handle files, as it automatically ensures the file is closed properly, even if exceptions occur.

## Question 17: Conceptual

What is a **generator** in Python, and what are its advantages?

**Answer:** A **generator** in Python is a special type of iterator that allows you to iterate over a sequence of values, but unlike lists, it does not store all the values in memory at once. Instead, it generates values on the fly, one at a time, using the `yield` keyword.

### Advantages of Generators:

- **Memory Efficiency:** Generators are highly memory efficient, especially when dealing with large datasets, because they produce items one by one and don't store the entire sequence in memory.
- **Lazy Evaluation:** Values are generated only when requested, which can improve performance for large or infinite sequences.
- **Readability:** They can make code more readable and concise for certain types of iterative tasks.

## Question 18: Coding

Write a Python function `fibonacci_sequence(n)` that generates the first `n` Fibonacci numbers using a generator.

**Answer:**

```
def fibonacci_sequence(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

# Test cases
print(list(fibonacci_sequence(0))) # Output: []
print(list(fibonacci_sequence(1))) # Output: [0]
print(list(fibonacci_sequence(7))) # Output: [0, 1, 1, 2, 3, 5, 8]
```

**Explanation:** The `fibonacci_sequence` function uses `yield` to return each Fibonacci number. It maintains two variables, `a` and `b`, to calculate the next number in the sequence. The loop runs `n` times, yielding `a` and then updating `a` and `b` for the next iteration.

## Question 19: Output Prediction

What will be the output of the following Python code?

```
numbers = [1, 2, 3, 4, 5]
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

**Answer:**

```
[2, 4]
```

**Explanation:** The `filter()` function constructs an iterator from elements of an iterable for which a function returns true. Here, the `lambda x: x % 2 == 0` function checks if a number is even. So, `filter()` selects only the even numbers from the `numbers` list, and `list()` converts the filter object into a list.

## Question 20: Debugging/Correction

The following code attempts to swap two variables, but it's not doing it correctly in a common scenario. Correct it using a Pythonic approach.

```
a = 5
b = 10

temp = a
a = b
b = temp

print(f"a: {a}, b: {b}")
```

### Corrected Code (Pythonic Swap):

```
a = 5
b = 10

a, b = b, a # Pythonic way to swap variables

print(f"a: {a}, b: {b}") # Output: a: 10, b: 5
```

**Explanation:** While the original code correctly swaps the variables, Python offers a more concise and readable way to swap values using tuple assignment. `a, b = b, a` simultaneously assigns the value of `b` to `a` and the value of `a` to `b`.

## Question 21: Conceptual

What is the difference between `==` and `is` operators in Python?

**Answer:** Both `==` and `is` are comparison operators in Python, but they check for different things:

- The `==` operator checks for **value equality**. It evaluates whether the values of two operands are equal.
- The `is` operator checks for **identity**. It evaluates whether two operands refer to the *same object* in memory.

### Example:

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = list1

print(list1 == list2) # Output: True (values are equal)
print(list1 is list2) # Output: False (different objects in memory)
print(list1 is list3) # Output: True (both refer to the same object)
```

### Question 22: Coding

Write a Python function `remove_duplicates(lst)` that takes a list `lst` and returns a new list with all duplicate elements removed, preserving the original order of the first occurrence.

### Answer:

```
def remove_duplicates(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

# Test cases
print(remove_duplicates([1, 2, 2, 3, 4, 4, 5])) # Output: [1, 2, 3, 4, 5]
print(remove_duplicates(['a', 'b', 'a', 'c', 'b'])) # Output: ['a', 'b', 'c']
```

**Explanation:** The function uses a `set` called `seen` to keep track of elements encountered so far. It iterates through the input list; if an element is not in `seen`, it's added to `seen` and appended to the `result` list. This ensures only the first occurrence of each element is kept.

## Question 23: Output Prediction

What will be the output of the following Python code?

```
def outer_function(x):  
    def inner_function(y):  
        return x + y  
    return inner_function  
  
closure = outer_function(10)  
print(closure(5))
```

**Answer:**

15

**Explanation:** This demonstrates a **closure**. `outer_function(10)` returns `inner_function`. The returned `inner_function` retains access to the `x` variable (which is 10) from its enclosing scope, even after `outer_function` has finished executing. When `closure(5)` is called, it executes `inner_function` with `y=5`, resulting in `10 + 5 = 15`.

## Question 24: Debugging/Correction

The following code intends to print numbers from 0 to 4, but it has an infinite loop. Correct it.

```
i = 0  
while i < 5:  
    print(i)  
    # Missing increment
```

**Corrected Code:**

```
i = 0
while i < 5:
    print(i)
    i += 1 # Increment i to eventually satisfy the loop condition
```

**Explanation:** The original code was missing the `i += 1` statement, which is necessary to increment the loop counter `i`. Without it, `i` would always remain 0, and the condition `i < 5` would always be true, leading to an infinite loop.

## Question 25: Conceptual

What is the purpose of the `pass` statement in Python?

**Answer:** The `pass` statement in Python is a null operation; nothing happens when it is executed. It is used as a placeholder where a statement is syntactically required but you don't want any code to execute.

**Common uses include:**

- **Placeholder for future code:** When you are designing a class, function, or loop, and you haven't decided on its implementation yet, you can use `pass` to avoid syntax errors.
- **Minimal classes/functions:** To create a minimal class or function that does nothing.

**Example:**

```
def my_function():
    pass # This function does nothing yet

class MyClass:
    pass # This class is empty for now

if True:
    pass # Do nothing if condition is true
```



## 2. Object-Oriented Programming (20 Questions)

---

### Question 26: Conceptual

What are the four main principles of Object-Oriented Programming (OOP), and briefly explain each?

**Answer:** The four main principles of Object-Oriented Programming (OOP) are:

1. **Encapsulation:** The bundling of data (attributes) and methods (functions) that operate on the data into a single unit, known as a class. It also involves restricting direct access to some of an object's components, preventing unintended interference and misuse of data. This is often achieved through access modifiers (though Python uses conventions like single or double underscores).
2. **Abstraction:** The concept of hiding the complex implementation details and showing only the essential features of an object. It focuses on what an object does rather than how it does it.
3. **Inheritance:** A mechanism where a new class (subclass/derived class) inherits properties and behaviors (attributes and methods) from an existing class (superclass/base class). This promotes code reusability and establishes a natural hierarchy between classes.
4. **Polymorphism:** The ability of objects of different classes to be treated as objects of a common superclass. It allows a single interface to be used for different data types or classes, leading to more flexible and extensible code. The word "polymorphism" means "many forms."

### Question 27: Coding

Create a `Dog` class with attributes `name` and `age`, and a method `bark()` that prints "Woof!". Instantiate two `Dog` objects and call their `bark()` method.

**Answer:**

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print("Woof!")

# Instantiate Dog objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)

# Access attributes
print(f"{dog1.name} is {dog1.age} years old.") # Output: Buddy is 3 years old.
print(f"{dog2.name} is {dog2.age} years old.") # Output: Lucy is 5 years old.

# Call methods
dog1.bark() # Output: Woof!
dog2.bark() # Output: Woof!
```

**Explanation:** The `Dog` class is defined with an `__init__` constructor to set `name` and `age`. The `bark` method simply prints “Woof!”. Two instances, `dog1` and `dog2`, are created, and their attributes are accessed, and the `bark()` method is called.

## Question 28: Output Prediction

What will be the output of the following Python code?

```
class Animal:
    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
    pass # Inherits speak from Animal

animal = Animal()
dog = Dog()
cat = Cat()

print(animal.speak())
print(dog.speak())
print(cat.speak())
```

### Answer:

```
Animal speaks
Woof
Animal speaks
```

### Explanation:

- `animal.speak()` calls the `speak` method from the `Animal` class.
- `dog.speak()` calls the overridden `speak` method from the `Dog` class.
- `cat.speak()` calls the `speak` method inherited from the `Animal` class because `Cat` does not override it.

### Question 29: Debugging/Correction

The following code intends to use a class variable to count the number of `car` objects created, but it has an error. Correct it.

```

class Car:
    count = 0 # Class variable

    def __init__(self, make, model):
        self.make = make
        self.model = model
        self.count += 1 # Incorrectly increments instance variable

car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

print(Car.count)

```

### Corrected Code:

```

class Car:
    count = 0 # Class variable

    def __init__(self, make, model):
        self.make = make
        self.model = model
        Car.count += 1 # Correctly increments the class variable

car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

print(Car.count) # Output: 2

```

**Explanation:** To correctly increment a class variable from within an instance method, you must refer to it using the class name (e.g., `Car.count`) instead of `self.count`. `self.count` would create and increment an instance-specific variable, not the shared class variable.

### Question 30: Conceptual

Explain the concept of **encapsulation** in Python and how it is typically achieved.

**Answer: Encapsulation** in Python refers to the practice of bundling data (attributes) and the methods that operate on that data within a single unit, a class. It also involves

controlling access to the internal state of an object, preventing direct modification from outside the class.

While Python does not have strict access modifiers like `public`, `private`, or `protected` as in some other languages, encapsulation is typically achieved through conventions:

- **Public attributes/methods:** By default, all attributes and methods in Python are public and can be accessed from anywhere.
- **Protected attributes/methods:** A single leading underscore (e.g., `_attribute`) indicates that an attribute or method is intended for internal use within the class or its subclasses. It's a convention, not an enforcement.
- **Private attributes/methods:** Double leading underscores (e.g., `__attribute`) trigger **name mangling**, making the attribute or method less directly accessible from outside the class. Python renames it internally (e.g., `_ClassName__attribute`), making it harder to accidentally overwrite or access. This provides a stronger form of encapsulation.

## Question 31: Coding

Create a `Shape` base class with an abstract method `area()`. Then create `Circle` and `Rectangle` subclasses that implement the `area()` method. Use the `abc` module for the abstract base class.

**Answer:**

```

from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius**2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

# Test cases
circle = Circle(5)
rectangle = Rectangle(4, 6)

print(f"Circle area: {circle.area()}") # Output: Circle area: 78.5398...
print(f"Rectangle area: {rectangle.area()}") # Output: Rectangle area: 24

# This would raise a TypeError: Can't instantiate abstract class Shape with
# abstract method area
# shape = Shape()

```

## Explanation:

- `Shape` is defined as an `ABC` (Abstract Base Class) with an `abstractmethod` `area()`. This means `Shape` cannot be instantiated directly, and any concrete subclass must implement `area()`.
- `Circle` and `Rectangle` inherit from `Shape` and provide their specific implementations of the `area()` method.

## Question 32: Output Prediction

What will be the output of the following Python code?

```
class A:
    def __init__(self):
        print("A init")

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")

class C(A):
    def __init__(self):
        super().__init__()
        print("C init")

class D(B, C):
    def __init__(self):
        super().__init__()
        print("D init")

d = D()
```

**Answer:**

```
A init
C init
B init
D init
```

**Explanation:** This demonstrates Python's Method Resolution Order (MRO) in multiple inheritance. When `super().__init__()` is called in `D`, it follows the MRO of `D`. The MRO for `D(B, C)` is `D -> B -> C -> A -> object`. So, `super().__init__()` in `D` calls `B.__init__()`, which in turn calls `super().__init__()` (which resolves to `C.__init__()` due to MRO), then `C.__init__()` calls `super().__init__()` (which resolves to `A.__init__()`). Finally, the `print` statements in each `__init__` are executed in reverse order of the `super()` calls.

## Question 33: Debugging/Correction

The following code intends to demonstrate polymorphism by having different animal classes speak, but the `make_sound` function is not working as expected. Correct it.

```
class Animal:
    def speak(self):
        return "Generic animal sound"

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

def make_sound(animal_object):
    # Missing call to speak method
    return animal_object

dog = Dog()
cat = Cat()

print(make_sound(dog))
print(make_sound(cat))
```

**Corrected Code:**



```

class Animal:
    def speak(self):
        return "Generic animal sound"

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

def make_sound(animal_object):
    return animal_object.speak() # Call the speak method

dog = Dog()
cat = Cat()

print(make_sound(dog)) # Output: Woof
print(make_sound(cat)) # Output: Meow

```

**Explanation:** The original `make_sound` function was simply returning the `animal_object` itself, not calling its `speak()` method. To achieve polymorphism, where the correct `speak()` method is called based on the object's actual type, you need to explicitly call `animal_object.speak()`.

## Question 34: Conceptual

What is a **class method** and a **static method** in Python, and how do they differ from instance methods?

**Answer:** In Python, **instance methods**, **class methods**, and **static methods** are different types of methods that can be defined within a class, each with a distinct purpose and way of receiving arguments:

- **Instance Method:**
  - Takes `self` as its first parameter, which refers to the instance of the class.
  - Can access and modify instance-specific data (attributes) and call other instance methods.

- Most common type of method.
- **Class Method:**
  - Decorated with `@classmethod`.
  - Takes `cls` as its first parameter, which refers to the class itself (not an instance).
  - Can access and modify class-level data (class variables) and call other class methods.
  - Often used for factory methods that create instances of the class.
- **Static Method:**
  - Decorated with `@staticmethod`.
  - Takes neither `self` nor `cls` as its first parameter.
  - Cannot access or modify instance-specific data or class-level data directly.
  - Behaves like a regular function but is logically associated with the class. It doesn't depend on the state of the object or the class.

### Key Differences:

- **Instance methods** operate on an instance and have access to instance data.
- **Class methods** operate on the class and have access to class data, often used for alternative constructors.
- **Static methods** are utility functions within the class that don't need access to either instance or class data.

### Question 35: Coding

Create a class `TemperatureConverter` with a class method `celsius_to_fahrenheit` and a static method `is_freezing`.

**Answer:**

```

class TemperatureConverter:
    @classmethod
    def celsius_to_fahrenheit(cls, celsius):
        return (celsius * 9/5) + 32

    @staticmethod
    def is_freezing(fahrenheit):
        return fahrenheit <= 32

# Using the class method
celsius_temp = 25
fahrenheit_temp = TemperatureConverter.celsius_to_fahrenheit(celsius_temp)
print(f"{celsius_temp}°C is {fahrenheit_temp}°F") # Output: 25°C is 77.0°F

# Using the static method
print(f"Is 30°F freezing? {TemperatureConverter.is_freezing(30)}") # Output:
Is 30°F freezing? True
print(f"Is 40°F freezing? {TemperatureConverter.is_freezing(40)}") # Output:
Is 40°F freezing? False

```

## Explanation:

- `celsius_to_fahrenheit` is a class method, taking `cls` as its first argument. It performs a conversion that doesn't depend on a specific instance's state but is related to the class's functionality.
- `is_freezing` is a static method. It's a utility function that logically belongs to the `TemperatureConverter` class but doesn't need access to `self` or `cls`.

## Question 36: Output Prediction

What will be the output of the following Python code?

```

class MyClass:
    def __init__(self, value):
        self.__private_value = value

    def get_private_value(self):
        return self.__private_value

obj = MyClass(10)
# print(obj.__private_value) # This line would cause an AttributeError
print(obj.get_private_value())
print(obj._MyClass__private_value) # Accessing mangled name

```

**Answer:**

```

10
10

```

**Explanation:**

- `self.__private_value` uses name mangling. Direct access like `obj.__private_value` would result in an `AttributeError` (commented out). However, accessing it through the public method `get_private_value()` works as intended.
- Python renames `__private_value` to `_MyClass__private_value` internally. Therefore, `obj._MyClass__private_value` can still access the mangled name, though this is generally discouraged as it bypasses the intended encapsulation.

## Question 37: Debugging/Correction

The following code intends to use a property to manage an employee's salary, but it has an issue with setting a negative salary. Correct it.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = salary # Using a
private-like attribute

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, new_salary):
        # Missing validation for negative salary
        self._salary = new_salary

emp = Employee("Alice", 50000)
print(f"Initial salary: {emp.salary}")

emp.salary = -1000 # This should be prevented
print(f"New salary: {emp.salary}")
```

**Corrected Code:**

```

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = 0 # Initialize to a safe value
        self.salary = salary # Use the setter for initial validation

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, new_salary):
        if new_salary >= 0:
            self._salary = new_salary
        else:
            print("Error: Salary cannot be negative.")

emp = Employee("Alice", 50000)
print(f"Initial salary: {emp.salary}") # Output: Initial salary: 50000

emp.salary = -1000 # This will now print an error and not change the salary
print(f"New salary: {emp.salary}") # Output: New salary: 50000 (salary
remains unchanged)

emp.salary = 60000
print(f"Updated salary: {emp.salary}") # Output: Updated salary: 60000

```

**Explanation:** The original code lacked validation in the `@salary.setter`. To prevent negative salaries, a condition `if new_salary >= 0:` was added. If the `new_salary` is negative, an error message is printed, and the `_salary` attribute is not updated. Additionally, the `__init__` method was modified to use the setter for initial salary assignment, ensuring validation applies from the start.

## Question 38: Conceptual

What is **method overriding** in Python OOP, and how does it relate to inheritance?

**Answer: Method overriding** in Python OOP is a feature that allows a subclass to provide a specific implementation for a method that is already defined in its superclass. When a method in a subclass has the same name, same parameters, and same return type (though Python doesn't strictly enforce return types) as a method in

its superclass, the method in the subclass is said to override the method in the superclass.

**Relation to Inheritance:** Method overriding is a direct consequence of inheritance. It enables **polymorphism**, allowing objects of different classes (a superclass and its subclasses) to respond differently to the same method call. When you call an overridden method on an object, Python's runtime determines which implementation to use based on the actual type of the object, not the type of the reference variable.

### Question 39: Coding

Create a base class `Vehicle` with a method `start_engine()`. Then create a subclass `Car` that overrides `start_engine()` to provide a car-specific message.

**Answer:**

```
class Vehicle:
    def start_engine(self):
        return "Engine started: Generic vehicle sound."

class Car(Vehicle):
    def start_engine(self):
        return "Engine started: Vroom vroom!"

vehicle = Vehicle()
car = Car()

print(vehicle.start_engine()) # Output: Engine started: Generic vehicle
                              sound.
print(car.start_engine())     # Output: Engine started: Vroom vroom!
```

**Explanation:** The `Car` class inherits from `Vehicle`. It then defines its own `start_engine()` method, which has the same name as the one in `Vehicle`. This new implementation in `Car` **overrides** the parent's method, so when `car.start_engine()` is called, the `car`'s specific implementation is executed.

### Question 40: Output Prediction

What will be the output of the following Python code?

```

class Parent:
    def __init__(self):
        self.value = "Parent Value"

class Child(Parent):
    def __init__(self):
        # super().__init__() is missing
        self.value = "Child Value"

p = Parent()
c = Child()

print(p.value)
print(c.value)

```

**Answer:**

```

Parent Value
Child Value

```

**Explanation:**

- When `p = Parent()` is created, `Parent.__init__` is called, setting `p.value` to "Parent Value".
- When `c = Child()` is created, `Child.__init__` is called. Because `super().__init__()` is *not* called in `Child.__init__`, the `Parent`'s `__init__` method is not executed. Therefore, `self.value` is directly set to "Child Value" within the `Child`'s constructor, without inheriting or calling the parent's initialization logic.

## Question 41: Debugging/Correction

The following code intends to use a `__str__` method for a custom string representation of a `Book` object, but it's not being called when printing the object directly. Correct it.



```

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __repr__(self):
        return f"Book(title='{self.title}', author='{self.author}')"

    # Missing __str__ method

book = Book("1984", "George Orwell")
print(book)

```

### Corrected Code:

```

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __repr__(self):
        return f"Book(title='{self.title}', author='{self.author}')"

    def __str__(self):
        return f"'{self.title}' by {self.author}"

book = Book("1984", "George Orwell")
print(book) # Output: '1984' by George Orwell

```

**Explanation:** When `print(book)` is called, Python looks for the `__str__` method to get a user-friendly string representation of the object. If `__str__` is not defined, it falls back to `__repr__`. By adding a `__str__` method, we provide the desired custom string output for `print()`.

### Question 42: Conceptual

What is the purpose of the `super()` function in Python, especially in the context of inheritance?

**Answer:** The `super()` function in Python is used to call a method from a parent or sibling class. It provides a way to access methods in the parent class that have been overridden in the child class. Its primary purpose is to enable cooperative multiple inheritance, ensuring that methods are called in a proper Method Resolution Order (MRO).

**In the context of inheritance:**

- It is most commonly used in the `__init__` method of a subclass to call the `__init__` method of its parent class, ensuring that the parent class's attributes are properly initialized.
- It allows you to extend the functionality of a parent method rather than completely replacing it, by calling the parent's method and then adding subclass-specific logic.

**Syntax:** `super().__init__(*args, **kwargs)` or `super().method_name(*args, **kwargs)`

### Question 43: Coding

Create a class `Person` with `name` and `age`. Then create a subclass `Student` that inherits from `Person` and adds an `id_number` attribute. Ensure `Student`'s `__init__` correctly calls the parent's `__init__`.

**Answer:**

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"Hi, my name is {self.name} and I am {self.age} years old."

class Student(Person):
    def __init__(self, name, age, id_number):
        super().__init__(name, age) # Call the parent's __init__ method
        self.id_number = id_number

    def introduce(self):
        # Override and extend parent's introduce method
        parent_intro = super().introduce()
        return f"{parent_intro} My student ID is {self.id_number}."

# Test cases
person = Person("Alice", 30)
student = Student("Bob", 20, "S12345")

print(person.introduce()) # Output: Hi, my name is Alice and I am 30 years old.
print(student.introduce()) # Output: Hi, my name is Bob and I am 20 years old. My student ID is S12345.

```

## Explanation:

- The `Student` class inherits from `Person`.
- In `Student`'s `__init__`, `super().__init__(name, age)` is called to initialize the `name` and `age` attributes using the `Person` class's constructor.
- The `id_number` is then initialized specifically for the `Student` class.
- The `introduce` method in `Student` also uses `super().introduce()` to get the parent's introduction and then extends it with student-specific information.

## Question 44: Output Prediction

What will be the output of the following Python code?

```
class A:
    def foo(self):
        print("A's foo")

class B(A):
    def foo(self):
        print("B's foo")

class C(A):
    def foo(self):
        print("C's foo")

class D(B, C):
    pass

d_obj = D()
d_obj.foo()
```

**Answer:**

```
B's foo
```

**Explanation:** This again relates to Python's Method Resolution Order (MRO). For `D(B, C)`, the MRO is `D -> B -> C -> A -> object`. When `d_obj.foo()` is called, Python searches for `foo` in `D`, then `B`, then `C`, then `A`. It finds `foo` in `B` first, so `B`'s implementation is executed.

## Question 45: Debugging/Correction

The following code attempts to create a singleton class, but it's not correctly enforcing the single instance. Correct it.

```

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        # Missing check for existing instance
        cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()

print(s1 is s2)

```

### Corrected Code:

```

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None: # Check if an instance already exists
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()

print(s1 is s2) # Output: True

```

**Explanation:** The `__new__` method is responsible for creating a new instance of a class. To implement a singleton pattern, we need to ensure that only one instance is ever created. The corrected code adds a check: if `_instance` is `None` (meaning no instance exists yet), it creates a new one; otherwise, it returns the existing `_instance`. This ensures that `s1` and `s2` refer to the exact same object.

### Question 46: Conceptual

What are **magic methods** (or dunder methods) in Python, and provide an example of their use.

**Answer: Magic methods**, also known as **dunder methods** (due to their double underscores, e.g., `__init__`, `__str__`), are special methods in Python that allow you to define how objects of your class behave in certain situations. They are not meant to be called directly by you but are invoked automatically by Python in response to specific operations or functions.

They enable Python's built-in functions and operators to work with custom objects, allowing you to define custom behavior for:

- Object initialization (`__init__`)
- String representation (`__str__`, `__repr__`)
- Arithmetic operations (`__add__`, `__sub__`)
- Comparison operations (`__eq__`, `__lt__`)
- Context management (`__enter__`, `__exit__`)
- And many more.

**Example (using `__add__` for custom addition):**

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2 # This calls v1.__add__(v2)
print(v3)    # Output: Vector(4, 6)
```

## Question 47: Coding

Create a class `wallet` that encapsulates a `balance`. Implement methods to `deposit` and `withdraw` money. Ensure the balance cannot go below zero.

## Answer:

```
class Wallet:
    def __init__(self, initial_balance=0):
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative.")
        self._balance = initial_balance # Use _balance for internal storage

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive.")
        self._balance += amount
        print(f"Deposited {amount}. New balance: {self._balance}")

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive.")
        if self._balance - amount < 0:
            print("Error: Insufficient funds.")
            return False
        self._balance -= amount
        print(f"Withdrew {amount}. New balance: {self._balance}")
        return True

# Test cases
my_wallet = Wallet(100)
print(f"Current balance: {my_wallet.balance}") # Output: Current balance:
100

my_wallet.deposit(50) # Output: Deposited 50. New balance: 150
my_wallet.withdraw(30) # Output: Withdrew 30. New balance: 120
my_wallet.withdraw(200) # Output: Error: Insufficient funds.
print(f"Current balance: {my_wallet.balance}") # Output: Current balance:
120

# Test initial negative balance (will raise ValueError)
# try:
#     bad_wallet = Wallet(-50)
# except ValueError as e:
#     print(e)
```

## Explanation:

- The `wallet` class uses a private-like attribute `_balance` to store the balance, accessed via a `@property` getter.
- The `deposit` method adds to the balance after validating a positive amount.
- The `withdraw` method subtracts from the balance only if sufficient funds are available, preventing the balance from going below zero.
- Initial balance also has a validation.

## Question 48: Output Prediction

What will be the output of the following Python code?

```
class A:
    def method(self):
        print("Method from A")

class B(A):
    pass

class C(A):
    def method(self):
        print("Method from C")

class D(B, C):
    pass

d_obj = D()
d_obj.method()
```

## Answer:

```
Method from B
```

**Explanation:** This is a trick question! The MRO for `D(B, C)` is `D -> B -> C -> A -> object`. However, `B` does not override `method()`. So, when `d_obj.method()` is



called, Python looks in `D` (not found), then `B` (not found), then `C` (found!). Therefore, `C`'s method is called. The output will be `Method from C`.

**Correction to my own explanation:** My previous explanation was incorrect. Let's re-evaluate the MRO and method lookup. The MRO for `D(B, C)` is indeed `D -> B -> C -> A -> object`. Since `B` does not define `method`, the lookup continues to `C`, where `method` is found. So the output should be `Method from C`.

```
# Let's verify the MRO
# print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class
'__main__.A'>, <class 'object'>)
```

## Question 49: Debugging/Correction

The following code attempts to use a `__del__` method to print a message when an object is destroyed, but it might not always behave as expected due to garbage collection timing. Correct it to reliably show object destruction for demonstration purposes.

```
class MyObject:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} created.")

    def __del__(self):
        print(f"{self.name} destroyed.")

obj1 = MyObject("Object 1")
obj2 = MyObject("Object 2")

# Deletion might not happen immediately or in predictable order
# del obj1
# del obj2
```

**Corrected Code (for demonstration purposes):**

```

import gc

class MyObject:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} created.")

    def __del__(self):
        print(f"{self.name} destroyed.")

obj1 = MyObject("Object 1")
obj2 = MyObject("Object 2")

# Explicitly delete objects and force garbage collection for demonstration
del obj1
del obj2
gc.collect() # Force garbage collection

print("End of script.")

```

**Explanation:** The `__del__` method (destructor) is called when an object's reference count drops to zero and it's about to be garbage collected. However, Python's garbage collector doesn't always run immediately or predictably. To reliably demonstrate `__del__` being called, especially in short scripts, you can explicitly `del` the objects to reduce their reference counts and then call `gc.collect()` to force a garbage collection cycle. In real-world applications, relying on `__del__` for critical resource management is generally discouraged; context managers (`with` statement) are preferred.

## Question 50: Conceptual

What is **composition** in OOP, and how does it differ from inheritance?

**Answer: Composition** is an OOP principle where a class contains instances of other classes as attributes. It represents a "has-a" relationship, meaning one object is composed of or has another object. Instead of inheriting behavior, a class delegates responsibilities to its contained objects.

**Difference from Inheritance:**

- **Inheritance (“is-a” relationship):** A subclass `is-a` type of its superclass (e.g., a `Car is-a Vehicle`). It promotes code reuse by inheriting methods and attributes.
- **Composition (“has-a” relationship):** A class `has-a` component object (e.g., a `Car has-a Engine`). It promotes code reuse by delegating tasks to the component objects. Composition offers greater flexibility because the components can be changed at runtime, and it avoids the tight coupling and potential complexities of deep inheritance hierarchies.

#### When to use:

- **Inheritance:** When there is a clear “is-a” relationship and you want to reuse implementation from a base class.
  - **Composition:** When there is a “has-a” relationship, and you want to build complex objects from simpler ones, promoting flexibility and loose coupling.
- 

## 3. NumPy (15 Questions)

---

### Question 51: Conceptual

What is NumPy, and what is its primary advantage over standard Python lists for numerical operations?

**Answer: NumPy** (Numerical Python) is a fundamental library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays.

**Primary Advantage over Standard Python Lists:** NumPy’s primary advantage lies in its **performance** and **memory efficiency** for numerical operations. This is due to:

- **Homogeneous Data:** NumPy arrays (ndarrays) store elements of the same data type, allowing for more efficient storage and operations.
- **C-implementation:** The core of NumPy is implemented in C, making operations much faster than Python’s native list operations, which involve Python objects.
- **Vectorization:** NumPy allows you to perform operations on entire arrays at once (vectorized operations) without explicit Python loops. This eliminates the

overhead of Python's interpreter for each element, leading to significant speedups.

- **Memory Locality:** NumPy arrays are stored in contiguous blocks of memory, which improves cache utilization and overall performance.

## Question 52: Coding

Create a NumPy array `arr` from the Python list `[1, 2, 3, 4, 5]`. Then, multiply all elements by 2 and print the result.

**Answer:**

```
import numpy as np

python_list = [1, 2, 3, 4, 5]
arr = np.array(python_list)

result = arr * 2
print(result) # Output: [ 2  4  6  8 10]
```

**Explanation:** `np.array()` converts the Python list into a NumPy array. The multiplication `arr * 2` is a vectorized operation, meaning it applies the multiplication to each element of the array efficiently without needing a loop.

## Question 53: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

**Answer:**

```
(2, 3)
2
6
```

### Explanation:

- `arr.shape` returns a tuple indicating the dimensions of the array. Here, it's a 2x3 array (2 rows, 3 columns).
- `arr.ndim` returns the number of dimensions (axes) of the array. A 2D array has 2 dimensions.
- `arr.size` returns the total number of elements in the array ( $2 * 3 = 6$ ).

## Question 54: Debugging/Correction

The following NumPy code intends to create an array of zeros with shape (3, 4), but it has a syntax error. Correct it.

```
import numpy as np

# zeros_array = np.zeros(3, 4) # Incorrect way to specify shape
# print(zeros_array)
```

### Corrected Code:

```
import numpy as np

zeros_array = np.zeros((3, 4)) # Shape must be passed as a tuple
print(zeros_array)
```

**Explanation:** When creating NumPy arrays with functions like `np.zeros()`, `np.ones()`, or `np.empty()`, the `shape` argument must be passed as a tuple (e.g., `(3, 4)`), not as separate arguments.

## Question 55: Conceptual

Explain the concept of **broadcasting** in NumPy with a simple example.

**Answer: Broadcasting** in NumPy is a powerful mechanism that allows NumPy to perform operations on arrays of different shapes. It implicitly allows NumPy to perform operations on arrays of different shapes during arithmetic operations. It effectively stretches the smaller array across the larger array so that they have compatible shapes.

### Broadcasting Rules:

1. If the arrays do not have the same number of dimensions, the shape of the smaller array is padded with ones on its left side.
2. If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
3. If in any dimension, the sizes are unequal and neither is 1, an error is raised.

### Example:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]]) # Shape (2, 3)
scalar = 10 # Treated as shape (1,)

result = arr + scalar # Scalar is broadcast across the entire array
print(result)
# Output:
# [[11 12 13]
#  [14 15 16]]

vector = np.array([10, 20, 30]) # Shape (3,)
result2 = arr + vector # Vector is broadcast row-wise
print(result2)
# Output:
# [[11 22 33]
#  [14 25 36]]
```

**Explanation:** In the first example, the scalar `10` is broadcast to every element of `arr`. In the second example, the 1D `vector` of shape `(3,)` is broadcast across the rows of

the `arr` (shape `(2, 3)`), effectively adding `[10, 20, 30]` to each row.

## Question 56: Coding

Create a 3x3 NumPy array with random integers between 1 and 10. Then, find the sum of each row and the mean of each column.

**Answer:**

```
import numpy as np

# Create a 3x3 array with random integers between 1 and 10
arr = np.random.randint(1, 11, size=(3, 3))
print("Original array:\n", arr)

# Sum of each row (axis=1)
row_sums = np.sum(arr, axis=1)
print("\nSum of each row:", row_sums)

# Mean of each column (axis=0)
col_means = np.mean(arr, axis=0)
print("Mean of each column:", col_means)
```

**Explanation:**

- `np.random.randint(1, 11, size=(3, 3))` creates a 3x3 array with random integers from 1 (inclusive) to 11 (exclusive).
- `np.sum(arr, axis=1)` calculates the sum along `axis=1` (rows).
- `np.mean(arr, axis=0)` calculates the mean along `axis=0` (columns).

## Question 57: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.arange(10).reshape(2, 5)
print(arr[0, 2])
print(arr[:, 1])
print(arr[1, :3])
```

**Answer:**

```
2
[1 6]
[5 6 7]
```

**Explanation:**

- `np.arange(10)` creates an array `[0, 1, ..., 9]`. `reshape(2, 5)` turns it into a 2x5 array.

```
[[0, 1, 2, 3, 4],
 [5, 6, 7, 8, 9]]
```

- `arr[0, 2]` accesses the element at row 0, column 2, which is `2`.
- `arr[:, 1]` accesses all rows (`:`) and column 1, resulting in `[1, 6]`.
- `arr[1, :3]` accesses row 1 and columns from 0 up to (but not including) 3, resulting in `[5, 6, 7]`.

## Question 58: Debugging/Correction

The following NumPy code intends to stack two 1D arrays vertically, but it uses an incorrect function. Correct it.



```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# stacked_arr = np.hstack((arr1, arr2)) # Incorrect for vertical stacking
# print(stacked_arr)
```

### Corrected Code:

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

stacked_arr = np.vstack((arr1, arr2)) # Correct function for vertical
stacking
print(stacked_arr)
# Output:
# [[1 2 3]
#  [4 5 6]]
```

**Explanation:** `np.hstack()` (horizontal stack) concatenates arrays column-wise. For vertical stacking (row-wise), `np.vstack()` should be used. Both functions take a tuple of arrays as input.

## Question 59: Conceptual

What is the purpose of `np.where()` in NumPy, and how is it used?

**Answer:** `np.where()` is a powerful NumPy function that returns elements chosen from `x` or `y` depending on `condition`. It is essentially a vectorized version of the `if-else` statement, allowing for efficient conditional selection of array elements.

**Usage:** `np.where(condition, x, y)`

- `condition`: A boolean array or array-like object. Where `True`, yield `x`, otherwise yield `y`.

- `x, y`: Arrays or scalars. The elements to choose from.

If only `condition` is provided, `np.where()` returns the indices of elements that are `True`.

### Example:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Replace values greater than 30 with 0
result = np.where(arr > 30, 0, arr)
print(result) # Output: [10 20 30  0  0]

# Get indices where condition is True
indices = np.where(arr > 30)
print(indices) # Output: (array([3, 4]),)
```

## Question 60: Coding

Given a NumPy array `data = np.array([1, 5, 2, 8, 3, 9, 4])`, use boolean indexing to select all elements greater than 4.

### Answer:

```

import numpy as np

data = np.array([1, 5, 2, 8, 3, 9, 4])

# Create a boolean array
condition = data > 4
print("Condition array:", condition) # Output: Condition array: [False  True
False  True False  True False]

# Use boolean indexing to select elements
selected_elements = data[condition]
print("Selected elements:", selected_elements) # Output: Selected elements:
[5 8 9]

# Alternatively, in one line:
selected_elements_online = data[data > 4]
print("Selected elements (one line):", selected_elements_online) # Output:
Selected elements (one line): [5 8 9]

```

**Explanation:** Boolean indexing involves creating a boolean array of the same shape as the original array. Where the boolean array is `True`, the corresponding element from the original array is selected. `data > 4` directly creates this boolean array.

## Question 61: Output Prediction

What will be the output of the following NumPy code?

```

import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr + 5)
print(arr * np.array([1, 0, 1, 0, 1]))

```

**Answer:**

```

[ 6  7  8  9 10]
[1 0 3 0 5]

```

## Explanation:

- `arr + 5` demonstrates broadcasting: the scalar `5` is added to each element of `arr`.
- `arr * np.array([1, 0, 1, 0, 1])` performs element-wise multiplication between the two arrays, as they have compatible shapes.

## Question 62: Debugging/Correction

The following NumPy code intends to calculate the dot product of two matrices, but it uses an incorrect operator for matrix multiplication. Correct it.

```
import numpy as np

matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# dot_product = matrix1 * matrix2 # Incorrect for matrix multiplication
# print(dot_product)
```

## Corrected Code:

```
import numpy as np

matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Correct way to perform matrix multiplication
dot_product = np.dot(matrix1, matrix2) # Using np.dot()
# Or using the @ operator (Python 3.5+)
# dot_product = matrix1 @ matrix2

print(dot_product)
# Output:
# [[19 22]
#  [43 50]]
```

**Explanation:** In NumPy, the `*` operator performs element-wise multiplication (Hadamard product), not matrix multiplication. For matrix multiplication, you should

use `np.dot()` or the `@` operator (available in Python 3.5 and later).

### Question 63: Conceptual

What is the difference between `np.array()` and `np.asarray()` ?

**Answer:** Both `np.array()` and `np.asarray()` are used to convert input into a NumPy array, but they have a subtle yet important difference:

- `np.array()` : Always creates a **new array** (a copy) from the input data, even if the input is already a NumPy array of the same `dtype` .
- `np.asarray()` : Creates a new array **only if** the input is not already a NumPy array of the same `dtype` . If the input is already an `ndarray` and its `dtype` matches, it returns a reference to the original array (no copy is made).

**When to use which:**

- Use `np.array()` when you explicitly need a **copy** of the data, ensuring that modifications to the new array do not affect the original data source.
- Use `np.asarray()` when you want to ensure the input is an array but want to avoid unnecessary copying if it's already an array. This can be more memory-efficient and faster when dealing with large arrays.

### Question 64: Coding

Create a NumPy array `arr = np.array([10, 20, 30, 40, 50])`. Use NumPy functions to find the maximum value, minimum value, and their respective indices.

**Answer:**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

max_value = np.max(arr)
min_value = np.min(arr)

max_index = np.argmax(arr)
min_index = np.argmin(arr)

print(f"Array: {arr}")
print(f"Maximum value: {max_value} at index: {max_index}") # Output: Maximum
value: 50 at index: 4
print(f"Minimum value: {min_value} at index: {min_index}") # Output: Minimum
value: 10 at index: 0
```

### Explanation:

- `np.max()` and `np.min()` return the maximum and minimum values in the array, respectively.
- `np.argmax()` and `np.argmin()` return the indices of the first occurrence of the maximum and minimum values, respectively.

### Question 65: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = arr.reshape(2, 3)
print(reshaped_arr)
print(reshaped_arr.flatten())
```

### Answer:

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

### Explanation:

- `arr.reshape(2, 3)` changes the shape of the 1D array `arr` into a 2x3 2D array.
- `reshaped_arr.flatten()` returns a copy of the array collapsed into one dimension.

---

## 4. Pandas (25 Questions)

---

### Question 66: Conceptual

What is Pandas, and what are its two primary data structures?

**Answer: Pandas** is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation library for Python. It is built on top of NumPy and provides data structures and operations for manipulating numerical tables and time series.

Its two primary data structures are:

1. **Series:** A one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It is similar to a column in a spreadsheet or a SQL table, or a dictionary with an ordered index.
2. **DataFrame:** A two-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet, SQL table, or a dictionary of Series objects. It is the most commonly used Pandas object.

### Question 67: Coding

Create a Pandas Series from a Python list `[10, 20, 30, 40, 50]` with custom index labels `["a", "b", "c", "d", "e"]`. Then, access the element at index "c".

**Answer:**

```
import pandas as pd

data = [10, 20, 30, 40, 50]
index_labels = ["a", "b", "c", "d", "e"]

series = pd.Series(data, index=index_labels)
print("Series:\n", series)

# Access element at index "c"
element_c = series["c"]
print("\nElement at index \"c\":", element_c) # Output: Element at index
"c": 30
```

### Explanation:

- `pd.Series()` is used to create a Series, passing the data and custom index labels.
- Elements can be accessed using their label-based index, similar to dictionary key access.

### Question 68: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
print(df['Age'].mean())
print(df[df['Age'] > 28])
```

### Answer:



30.0

|   | Name    | Age | City        |
|---|---------|-----|-------------|
| 1 | Bob     | 30  | Los Angeles |
| 2 | Charlie | 35  | Chicago     |

### Explanation:

- `df['Age'].mean()` calculates the mean of the 'Age' column, which is  $(25 + 30 + 35) / 3 = 30.0$ .
- `df[df['Age'] > 28]` uses boolean indexing to select rows where the 'Age' is greater than 28. This selects the rows for 'Bob' and 'Charlie'.

### Question 69: Debugging/Correction

The following Pandas code intends to read a CSV file named `data.csv` into a DataFrame, but it might encounter a `FileNotFoundError`. Correct it to handle the case where the file doesn't exist.

```
import pandas as pd

# df = pd.read_csv('data.csv') # This line might cause an error
# print(df.head())
```

### Corrected Code:

```

import pandas as pd
import os

file_path = 'data.csv'

# Create a dummy CSV file for demonstration if it doesn't exist
if not os.path.exists(file_path):
    dummy_data = {
        'col1': [1, 2, 3],
        'col2': ['A', 'B', 'C']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv(file_path, index=False)
    print(f"Created dummy file: {file_path}")

try:
    df = pd.read_csv(file_path)
    print("DataFrame head:\n", df.head())
except FileNotFoundError:
    print(f"Error: The file {file_path} was not found.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

```

## Explanation:

- The original code would raise a `FileNotFoundError` if `data.csv` did not exist. The corrected code wraps `pd.read_csv()` in a `try-except` block to gracefully handle this specific error.
- For demonstration purposes, a dummy `data.csv` is created if it doesn't exist, so the code can run without manual file creation.

## Question 70: Conceptual

Explain the difference between `.loc` and `.iloc` in Pandas DataFrames.

**Answer:** Both `.loc` and `.iloc` are used for selecting data from Pandas DataFrames, but they differ in how they interpret the indexing arguments:

- `.loc` is primarily **label-based indexing**. It uses the actual row and column labels (names) to select data. It can also take boolean arrays.

- Syntax: `df.loc[row_label, column_label]`
- The end label in slicing is **inclusive**.
- `.iloc` is primarily **integer-location based indexing**. It uses the integer positions (0-based) of rows and columns to select data.
  - Syntax: `df.iloc[row_position, column_position]`
  - The end position in slicing is **exclusive** (like standard Python slicing).

### Example:

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])

print("DataFrame:\n", df)

# Using .loc
print("\n.loc[\"y\", \"B\"]:", df.loc["y", "B"]) # Output: 5
print("\n.loc[\"x\": \"y\", \"A\"]:\n", df.loc["x": "y", "A"]) # Output:
Series with 'x': 1, 'y': 2

# Using .iloc
print("\n.iloc[1, 1]:", df.iloc[1, 1]) # Output: 5
print("\n.iloc[0:2, 0]:\n", df.iloc[0:2, 0]) # Output: Series with 'x': 1,
'y': 2
```

## Question 71: Coding

Given a DataFrame `df` with columns 'A', 'B', 'C', calculate the sum of column 'A', the mean of column 'B', and the maximum value of column 'C'.

### Answer:

```

import pandas as pd

data = {
    'A': [10, 20, 30, 40],
    'B': [1, 2, 3, 4],
    'C': [100, 200, 50, 150]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

sum_A = df['A'].sum()
mean_B = df['B'].mean()
max_C = df['C'].max()

print(f"\nSum of column A: {sum_A}")    # Output: Sum of column A: 100
print(f"Mean of column B: {mean_B}")    # Output: Mean of column B: 2.5
print(f"Max of column C: {max_C}")      # Output: Max of column C: 200

```

**Explanation:** Pandas Series (which columns of a DataFrame are) have built-in methods like `.sum()`, `.mean()`, and `.max()` for common aggregations.

## Question 72: Output Prediction

What will be the output of the following Pandas code?

```

import pandas as pd

df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['A', 'B', 'C']
})
df.set_index('col2', inplace=True)
print(df.loc['B', 'col1'])

```

**Answer:**

2

## Explanation:

- A DataFrame is created.
- `df.set_index('col2', inplace=True)` sets the 'col2' column as the new index of the DataFrame, modifying it in place. The DataFrame now looks like:

```
      col1
col2
A        1
B        2
C        3
```

- `df.loc['B', 'col1']` then uses label-based indexing to select the value in the row labeled 'B' and column 'col1', which is 2.

## Question 73: Debugging/Correction

The following Pandas code intends to drop rows with any missing values, but it uses an incorrect method. Correct it.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8],
    'C': [9, 10, 11, 12]
})

# df_cleaned = df.fillna(0) # Incorrect for dropping rows
# print(df_cleaned)
```

## Corrected Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8],
    'C': [9, 10, 11, 12]
})
print("Original DataFrame:\n", df)

df_cleaned = df.dropna() # Correct method to drop rows with any NaN values
print("\nCleaned DataFrame (dropna):\n", df_cleaned)

# Output:
#    A    B    C
# 0  1.0  5.0    9
# 3  4.0  8.0   12
```

### Explanation:

- `df.fillna(0)` replaces `NaN` values with `0`, it does not remove rows. The original intent was to drop rows with missing values.
- `df.dropna()` is the correct method to remove rows (or columns) containing `NaN` (Not a Number) values. By default, it drops rows if *any* `NaN` is present.

## Question 74: Conceptual

What is the purpose of `groupby()` in Pandas, and provide a simple example.

**Answer:** The `groupby()` method in Pandas is used for grouping rows of a DataFrame together based on some criteria, typically one or more column values. It is a fundamental operation for performing aggregate analysis on subsets of data. The process generally involves three steps:

1. **Splitting:** The data is split into groups based on some criteria.
2. **Applying:** A function (e.g., aggregation, transformation, filtration) is applied to each group independently.
3. **Combining:** The results of these operations are combined into a new DataFrame or Series.

## Example:

```
import pandas as pd

data = {
    'Category': ['A', 'B', 'A', 'B', 'A'],
    'Value': [10, 20, 15, 25, 12]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Group by 'Category' and calculate the sum of 'Value' for each group
grouped_sum = df.groupby('Category')['Value'].sum()
print("\nGrouped Sum:\n", grouped_sum)
# Output:
# Category
# A      37
# B      45
# Name: Value, dtype: int64

# Group by 'Category' and calculate the mean of 'Value' for each group
grouped_mean = df.groupby('Category')['Value'].mean()
print("\nGrouped Mean:\n", grouped_mean)
# Output:
# Category
# A      12.333333
# B      22.500000
# Name: Value, dtype: float64
```

**Explanation:** In this example, the DataFrame is grouped by the 'Category' column. For each category ('A' and 'B'), the sum and mean of the 'Value' column are calculated, demonstrating how `groupby()` facilitates aggregate analysis.

## Question 75: Coding

Given two DataFrames, `df1` and `df2`, merge them based on a common column 'ID'.

**Answer:**

```

import pandas as pd

df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'ID': [2, 3, 4],
    'Score': [85, 90, 78]
})

print("DataFrame 1:\n", df1)
print("\nDataFrame 2:\n", df2)

# Merge DataFrames on the 'ID' column (inner join by default)
merged_df = pd.merge(df1, df2, on='ID')
print("\nMerged DataFrame (Inner Join):\n", merged_df)
# Output:
#    ID    Name  Score
# 0    2     Bob     85
# 1    3  Charlie     90

# Example of a left join
left_merged_df = pd.merge(df1, df2, on='ID', how='left')
print("\nMerged DataFrame (Left Join):\n", left_merged_df)
# Output:
#    ID    Name  Score
# 0    1   Alice   NaN
# 1    2     Bob   85.0
# 2    3  Charlie   90.0

```

**Explanation:** `pd.merge()` is used to combine DataFrames based on common columns or indices. By default, it performs an inner join, keeping only the rows where the 'ID' exists in both DataFrames. The `how` parameter can be used to specify other types of joins (e.g., 'left', 'right', 'outer').

## Question 76: Output Prediction

What will be the output of the following Pandas code?



```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.T)
```

**Answer:**

```
   0  1  2
A  1  2  3
B  4  5  6
```

**Explanation:** `df.T` (or `df.transpose()`) transposes the DataFrame, swapping rows and columns. The original column names ('A', 'B') become the new index, and the original index (0, 1, 2) becomes the new column names.

## Question 77: Debugging/Correction

The following Pandas code intends to apply a function to each element of a Series, but it uses a regular Python loop which is less efficient. Correct it to use a Pandas vectorized operation or `.apply()`.

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])

# result = []
# for x in s:
#     result.append(x * 2)
# new_s = pd.Series(result)
# print(new_s)
```

**Corrected Code (Vectorized Operation):**

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])
print("Original Series:\n", s)

# Vectorized operation (most efficient)
new_s_vectorized = s * 2
print("\nNew Series (vectorized):\n", new_s_vectorized)

# Output:
# 0      2
# 1      4
# 2      6
# 3      8
# 4     10
# dtype: int64
```

### Corrected Code (Using `.apply()` for more complex functions):

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])

def custom_function(x):
    return x * 2 + 1 # More complex operation

new_s_apply = s.apply(custom_function)
print("\nNew Series (apply):\n", new_s_apply)

# Output:
# 0      3
# 1      5
# 2      7
# 3      9
# 4     11
# dtype: int64
```

### Explanation:

- For simple element-wise operations like multiplication, Pandas Series support vectorized operations directly (e.g., `s * 2`), which are highly optimized and much faster than explicit Python loops.

- For more complex, custom functions, the `.apply()` method is the preferred way to apply a function along an axis of a DataFrame or Series. It is still significantly faster than a pure Python loop.

## Question 78: Conceptual

What is a MultiIndex (Hierarchical Index) in Pandas, and when is it useful?

**Answer:** A **MultiIndex**, also known as a **Hierarchical Index**, is a Pandas Index object that allows for multiple levels of indexing on an axis (rows or columns) of a Series or DataFrame. It provides a way to store and work with data that has more than one logical grouping or category.

### When it is useful:

- **Complex Data Structures:** When dealing with data that naturally has multiple levels of categorization, such as time-series data with year and month, or experimental data with different conditions and replicates.
- **Data Aggregation:** It simplifies operations like `groupby()` on multiple levels, allowing for more sophisticated data analysis.
- **Data Selection:** Provides powerful and flexible ways to select subsets of data using multiple index levels.

### Example:

```
import pandas as pd

# Create a MultiIndex DataFrame
index = pd.MultiIndex.from_product([['A', 'B'], [1, 2]], names=['Category',
'Subcategory'])
df = pd.DataFrame({
    'Value': [10, 20, 30, 40],
    'Data': ['x', 'y', 'z', 'w']
}, index=index)
print("MultiIndex DataFrame:\n", df)

# Output:
#
#           Value Data
# Category Subcategory
# A         1         10    x
#           2         20    y
# B         1         30    z
#           2         40    w

# Accessing data with MultiIndex
print("\nAccessing data for Category A:\n", df.loc['A'])

# Output:
#
#           Value Data
# Subcategory
# 1           10    x
# 2           20    y
```

## Question 79: Coding

Create a Pandas DataFrame from a dictionary of lists. Add a new column 'Total' which is the sum of 'Col1' and 'Col2'. Then, drop the 'Col2' column.

**Answer:**

```
import pandas as pd

data = {
    'Col1': [10, 20, 30],
    'Col2': [1, 2, 3],
    'Col3': ['A', 'B', 'C']
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Add 'Total' column
df['Total'] = df['Col1'] + df['Col2']
print("\nDataFrame after adding 'Total':\n", df)

# Drop 'Col2' column
df_modified = df.drop('Col2', axis=1) # axis=1 for columns, inplace=False by default
print("\nDataFrame after dropping 'Col2':\n", df_modified)

# Output:
#      Col1 Col3  Total
# 0      10   A      11
# 1      20   B      22
# 2      30   C      33
```

## Explanation:

- New columns can be added by simply assigning a Series (or a scalar) to a new column name.
- `df.drop()` is used to remove rows or columns. `axis=1` specifies that a column should be dropped. By default, `drop()` returns a new DataFrame; `inplace=True` would modify the original DataFrame.

## Question 80: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': ['P', 'Q', 'P', 'Q']
})

print(df.groupby('B')['A'].sum())
```

**Answer:**

```
B
P    4
Q    6
Name: A, dtype: int64
```

**Explanation:**

- The DataFrame is grouped by the unique values in column 'B' ('P' and 'Q').
- For each group, the sum of the corresponding values in column 'A' is calculated.
  - For 'P':  $1 + 3 = 4$
  - For 'Q':  $2 + 4 = 6$

## Question 81: Debugging/Correction

The following Pandas code intends to rename a column, but it's not correctly applying the change. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'old_name': [1, 2, 3],
    'B': [4, 5, 6]
})

# df.rename(columns={'old_name': 'new_name'}) # Does not modify in-place by default
# print(df.columns)
```

## Corrected Code:

```
import pandas as pd

df = pd.DataFrame({
    'old_name': [1, 2, 3],
    'B': [4, 5, 6]
})
print("Original columns:", df.columns)

# Correct way to rename a column (using inplace=True or reassigning)
df.rename(columns={'old_name': 'new_name'}, inplace=True)
# Alternatively, reassign:
# df = df.rename(columns={'old_name': 'new_name'})

print("New columns:", df.columns) # Output: New columns: Index(['new_name',
'B'], dtype='object')
```

**Explanation:** The `rename()` method, like many Pandas methods, returns a new DataFrame by default and does not modify the original DataFrame in-place unless `inplace=True` is specified. To apply the change, either reassign the result back to the DataFrame (`df = df.rename(...)`) or use `inplace=True`.

## Question 82: Conceptual

What is the purpose of `apply()` and `map()` in Pandas, and when would you use each?

**Answer:** Both `apply()` and `map()` are used to apply functions to Pandas Series or DataFrames, but they serve different purposes and are used in different contexts:

- **`.apply()` :**
  - **Purpose:** Applies a function along an axis of a DataFrame or to the elements of a Series. It is more general and can operate on entire rows/columns or element-wise.
  - **Usage:**
    - On a **Series**: Applies a function to each element.
    - On a **DataFrame**: Applies a function to each row (`axis=1`) or each column (`axis=0`). The function should expect a Series as input and

return a Series or a scalar.

- **When to use:** For more complex operations that cannot be achieved with vectorized NumPy functions, or when you need to operate on entire rows/columns.
- **.map()** :
  - **Purpose:** Element-wise mapping of values in a Series. It is used to substitute each value in a Series with another value, often from a dictionary or another Series.
  - **Usage:** Only works on a **Series**.
  - **When to use:** For substituting values based on a lookup table (dictionary or Series), or when you need to apply a simple element-wise transformation that can be expressed as a mapping.

### Example:

```
import pandas as pd

s = pd.Series([1, 2, 3, 4])

# Using .apply() on a Series
def square(x):
    return x**2
s_apply = s.apply(square)
print("Series with .apply():\n", s_apply)

# Using .map() on a Series with a dictionary
mapping_dict = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
s_map = s.map(mapping_dict)
print("\nSeries with .map():\n", s_map)
```

## Question 83: Coding

Given a DataFrame `df` with a 'Date' column (as strings in 'YYYY-MM-DD' format), convert it to datetime objects and then extract the year and month into new columns 'Year' and 'Month'.

**Answer:**



```

import pandas as pd

df = pd.DataFrame({
    'Date': ['2023-01-15', '2023-02-20', '2024-03-10'],
    'Value': [100, 150, 200]
})
print("Original DataFrame:\n", df)
print("Date column dtype before conversion:", df['Date'].dtype)

# Convert 'Date' column to datetime objects
df['Date'] = pd.to_datetime(df['Date'])
print("\nDate column dtype after conversion:", df['Date'].dtype)

# Extract Year and Month
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month

print("\nDataFrame with Year and Month:\n", df)
# Output:
#           Date  Value  Year  Month
# 0 2023-01-15    100   2023      1
# 1 2023-02-20    150   2023      2
# 2 2024-03-10    200   2024      3

```

### Explanation:

- `pd.to_datetime()` is used to convert a Series of strings into datetime objects. This is crucial for time-series operations.
- Once the column is of `datetime` type, the `.dt` accessor allows easy extraction of date and time components like `year`, `month`, `day`, `hour`, etc.

### Question 84: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.sum(axis=0))
print(df.sum(axis=1))
```

### Answer:

```
A      6
B     15
dtype: int64
0      5
1      7
2      9
dtype: int64
```

### Explanation:

- `df.sum(axis=0)` calculates the sum for each column (default behavior if `axis` is not specified). So, A:  $1+2+3=6$  , B:  $4+5+6=15$  .
- `df.sum(axis=1)` calculates the sum for each row. So, 0:  $1+4=5$  , 1:  $2+5=7$  , 2:  $3+6=9$  .

## Question 85: Debugging/Correction

The following Pandas code intends to fill missing values in a DataFrame with the mean of their respective columns, but it's not correctly handling the `inplace` parameter. Correct it.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8]
})

# df.fillna(df.mean(), inplace=False) # This would return a new DataFrame
# but not assign it
# print(df)
```

## Corrected Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8]
})
print("Original DataFrame:\n", df)

# Correct way to fill NaN with column means (using inplace=True or
# reassigning)
df.fillna(df.mean(), inplace=True)
# Alternatively, reassign:
# df = df.fillna(df.mean())

print("\nDataFrame after filling NaNs with column means:\n", df)
# Output:
#      A      B
# 0  1.0  5.0
# 1  2.0  6.666667
# 2  2.333333  7.0
# 3  4.0  8.0
```

**Explanation:** Similar to `rename()`, `fillna()` returns a new DataFrame by default. To modify the original DataFrame, you must either set `inplace=True` or reassign the result of `fillna()` back to the DataFrame variable.

## Question 86: Conceptual

What is a Pivot Table in Pandas, and what is its main use case?

**Answer:** A **Pivot Table** in Pandas is a powerful data summarization tool that allows you to reorganize and aggregate data from a DataFrame. It takes column values from the original DataFrame and transforms them into a new index (rows), columns, and values, often applying an aggregation function (like sum, mean, count) to the data.

**Main Use Case:** Pivot tables are primarily used for **summarizing and analyzing data by one or more categorical variables**. They are excellent for:

- **Multi-dimensional analysis:** Easily view data from different perspectives.
- **Reporting:** Creating summary reports from raw data.
- **Data exploration:** Quickly understand relationships and patterns within your data.

**Example:**

```
import pandas as pd

data = {
    'Region': ['East', 'West', 'East', 'West', 'East'],
    'Product': ['A', 'B', 'A', 'C', 'B'],
    'Sales': [100, 150, 120, 200, 130]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Create a pivot table to show total sales by Region and Product
pivot_table = df.pivot_table(values='Sales', index='Region',
                              columns='Product', aggfunc='sum')
print("\nPivot Table (Total Sales by Region and Product):\n", pivot_table)

# Output:
# Product      A      B      C
# Region
# East      220.0  130.0   NaN
# West       NaN  150.0  200.0
```

## Question 87: Coding

Given a DataFrame `df` with a 'Price' column, create a new column 'Price\_Category' based on the following conditions: 'High' if Price > 100, 'Medium' if Price is between 50 and 100 (inclusive), and 'Low' if Price < 50.

**Answer:**

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Item': ['A', 'B', 'C', 'D', 'E'],
    'Price': [120, 60, 30, 90, 150]
})
print("Original DataFrame:\n", df)

# Define conditions and choices
conditions = [
    (df['Price'] > 100),
    (df['Price'] >= 50) & (df['Price'] <= 100),
    (df['Price'] < 50)
]
choices = ['High', 'Medium', 'Low']

# Use np.select to create the new column
df['Price_Category'] = np.select(conditions, choices, default='Unknown')

print("\nDataFrame with Price_Category:\n", df)

# Output:
#   Item  Price Price_Category
# 0    A    120             High
# 1    B     60           Medium
# 2    C     30             Low
# 3    D     90           Medium
# 4    E    150             High
```

**Explanation:** `np.select()` is a vectorized way to choose values from a list of choices based on a list of conditions. It is more efficient than using multiple `if-elif-else` statements or `.apply()` for this type of conditional logic on DataFrames.

## Question 88: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

s = pd.Series([1, 2, None, 4, 5])
print(s.isnull().sum())
print(s.dropna())
```

**Answer:**

```
1
0    1.0
1    2.0
3    4.0
4    5.0
dtype: float64
```

**Explanation:**

- `s.isnull()` returns a boolean Series indicating where values are `NaN` (or `None`). `s.isnull().sum()` counts the number of `True` values, which is `1` (for the `None` value).
- `s.dropna()` returns a new Series with `NaN` values removed. The original index is preserved for the non-`NaN` values.

## Question 89: Debugging/Correction

The following Pandas code intends to sort a DataFrame by a column in descending order, but it's not correctly specifying the sorting order. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})

# df.sort_values(by='Score') # Sorts in ascending order by default
# print(df)
```

### Corrected Code:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})
print("Original DataFrame:\n", df)

# Correct way to sort in descending order
df_sorted = df.sort_values(by='Score', ascending=False)
print("\nSorted DataFrame (descending by Score):\n", df_sorted)

# Output:
#      Name  Score
# 1    Bob     92
# 0  Alice     85
# 2  Charlie    78
```

**Explanation:** By default, `sort_values()` sorts in ascending order. To sort in descending order, you need to set the `ascending` parameter to `False`.

### Question 90: Conceptual

What is the purpose of `pd.concat()` and `pd.merge()` in Pandas, and when would you use each?

**Answer:** Both `pd.concat()` and `pd.merge()` are used to combine DataFrames, but they do so in different ways and are suitable for different scenarios:

- `pd.concat()` :
  - **Purpose:** Concatenates Pandas objects (Series or DataFrames) along a particular axis (rows or columns). It essentially “stacks” or “joins” them side-by-side or one on top of another.
  - **When to use:** When you want to append DataFrames (add rows) or combine them side-by-side (add columns) without considering common columns/indices for alignment. It’s useful for combining data that has the same structure or when you want to stack data.
- `pd.merge()` :
  - **Purpose:** Combines DataFrames based on common columns or indices, similar to SQL joins. It aligns rows from two DataFrames based on matching values in specified key columns.
  - **When to use:** When you want to combine DataFrames that share one or more common columns (keys) and you need to align the data based on these keys. It’s ideal for relational operations.

**Example (pd.concat()):**



```

import pandas as pd

df1 = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})
df2 = pd.DataFrame({
    'A': [5, 6],
    'B': [7, 8]
})

# Concatenate along rows (default)
concat_rows = pd.concat([df1, df2])
print("\nConcatenated (rows):\n", concat_rows)
# Output:
#    A  B
# 0  1  3
# 1  2  4
# 0  5  7
# 1  6  8

# Concatenate along columns
df3 = pd.DataFrame({
    'C': [9, 10],
    'D': [11, 12]
})
concat_cols = pd.concat([df1, df3], axis=1)
print("\nConcatenated (columns):\n", concat_cols)
# Output:
#    A  B   C   D
# 0  1  3   9  11
# 1  2  4  10  12

```

**Example (pd.merge()):**

```
import pandas as pd

df_left = pd.DataFrame({
    'key': ['K0', 'K1', 'K2', 'K3'],
    'A': ['A0', 'A1', 'A2', 'A3'],
    'B': ['B0', 'B1', 'B2', 'B3']
})

df_right = pd.DataFrame({
    'key': ['K0', 'K1', 'K4', 'K5'],
    'C': ['C0', 'C1', 'C4', 'C5'],
    'D': ['D0', 'D1', 'D4', 'D5']
})

merged_df = pd.merge(df_left, df_right, on='key', how='inner')
print("\nMerged (inner join):\n", merged_df)
# Output:
#   key  A  B  C  D
# 0  K0  A0 B0 C0 D0
# 1  K1  A1 B1 C1 D1
```

## Question 91: Coding

Given a DataFrame `df` with a column 'Values', normalize the values to be between 0 and 1 (Min-Max Normalization).

**Answer:**

```
import pandas as pd

df = pd.DataFrame({
    'Values': [10, 20, 30, 40, 50]
})
print("Original DataFrame:\n", df)

min_val = df['Values'].min()
max_val = df['Values'].max()

df['Normalized_Values'] = (df['Values'] - min_val) / (max_val - min_val)

print("\nDataFrame with Normalized_Values:\n", df)
# Output:
#   Values  Normalized_Values
# 0     10             0.00
# 1     20             0.25
# 2     30             0.50
# 3     40             0.75
# 4     50             1.00
```

**Explanation:** Min-Max Normalization is calculated as  $(X - \min(X)) / (\max(X) - \min(X))$ . Pandas allows for vectorized operations on Series, making this calculation straightforward and efficient.

## Question 92: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.pipe(lambda x: x + 1))
```

**Answer:**

```
   A  B
0  2  5
1  3  6
2  4  7
```

**Explanation:** The `.pipe()` method allows you to chain together functions that expect a DataFrame or Series as input and return a DataFrame or Series. Here, a lambda function `lambda x: x + 1` is applied to the entire DataFrame, adding 1 to each element.

### Question 93: Debugging/Correction

The following Pandas code intends to reset the index of a DataFrame, but it's not correctly dropping the old index. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])

# df.reset_index() # Keeps the old index as a column by default
# print(df)
```

**Corrected Code:**

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
print("Original DataFrame:\n", df)

# Correct way to reset index and drop the old index
df_reset = df.reset_index(drop=True) # drop=True prevents old index from
becoming a column
print("\nDataFrame after resetting index:\n", df_reset)

# Output:
#      A  B
# 0    1  4
# 1    2  5
# 2    3  6
```

**Explanation:** By default, `reset_index()` converts the old index into a new column. To discard the old index entirely, you need to set the `drop` parameter to `True`.

## Question 94: Conceptual

What is the purpose of `value_counts()` in Pandas Series/DataFrames?

**Answer:** The `value_counts()` method in Pandas is used to return a Series containing counts of unique values. The resulting Series will be in descending order so that the first element is the most frequently occurring element. Excludes NA values by default.

**Main Use Case:** It is extremely useful for **exploratory data analysis (EDA)** to quickly understand the distribution of categorical data or to identify the frequency of different items in a column.

**Example:**

```
import pandas as pd

s = pd.Series(["apple", "banana", "apple", "orange", "banana", "apple"])
print("Original Series:\n", s)

value_counts = s.value_counts()
print("\nValue Counts:\n", value_counts)

# Output:
# apple      3
# banana     2
# orange     1
# dtype: int64
```

## Question 95: Coding

Given a DataFrame `df` with a 'Text' column, convert all text to lowercase and remove leading/trailing whitespace.

**Answer:**

```
import pandas as pd

df = pd.DataFrame({
    'Text': [' Hello World ', ' Python ', 'Pandas']
})
print("Original DataFrame:\n", df)

# Convert to lowercase and strip whitespace
df['Text_Cleaned'] = df['Text'].str.lower().str.strip()

print("\nDataFrame with Text_Cleaned:\n", df)

# Output:
#           Text Text_Cleaned
# 0  Hello World  hello world
# 1      Python    python
# 2      Pandas   pandas
```

**Explanation:** Pandas Series of string type have a special `.str` accessor that provides a rich set of string manipulation methods. `str.lower()` converts to lowercase, and `str.strip()` removes leading/trailing whitespace.

---

## 5. Matplotlib (15 Questions)

---

### Question 96: Conceptual

What is Matplotlib, and what is its primary purpose?

**Answer: Matplotlib** is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is widely used for generating plots, histograms, power spectra, bar charts, error charts, scatterplots, etc.

**Primary Purpose:** Its primary purpose is to **provide a flexible and powerful foundation for creating high-quality data visualizations** in Python. It allows users to produce publication-quality figures in a variety of hardcopy formats and interactive environments across platforms. It is often used for:

- **Exploratory Data Analysis (EDA):** To visualize data distributions, relationships, and patterns.
- **Reporting and Presentation:** To create clear and informative plots for reports, presentations, and academic papers.
- **Customization:** Offers extensive control over every aspect of a figure, from plot styles to text properties.

### Question 97: Coding

Create a simple line plot using Matplotlib with `x = [1, 2, 3, 4]` and `y = [10, 15, 13, 18]`. Add a title "Simple Line Plot" and label the x-axis "X-axis" and y-axis "Y-axis".

**Answer:**

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 15, 13, 18]

plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True) # Optional: Add a grid for better readability
plt.show()
```

### Explanation:

- `plt.plot(x, y)` creates the line plot.
- `plt.title()`, `plt.xlabel()`, and `plt.ylabel()` are used to add a title and axis labels.
- `plt.show()` displays the plot.

### Question 98: Output Prediction

What will be the output (in terms of plot elements) of the following Matplotlib code?

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.figure(figsize=(8, 4))
plt.plot(x, y1, label='sin(x)', color='blue', linestyle='--')
plt.plot(x, y2, label='cos(x)', color='red', linestyle='-')
plt.legend()
plt.show()
```

**Answer:** This code will display a plot with two lines. One line will represent `sin(x)` in blue with a dashed linestyle, and the other will represent `cos(x)` in red with a solid



linestyle. A legend will be displayed, showing the labels 'sin(x)' and 'cos(x)'. The figure will have a width of 8 inches and a height of 4 inches.

### Explanation:

- `plt.figure(figsize=(8, 4))` creates a figure with a specified size.
- Two `plt.plot()` calls are made, each with different `label`, `color`, and `linestyle` arguments.
- `plt.legend()` displays the legend using the labels provided in the `plt.plot()` calls.

### Question 99: Debugging/Correction

The following Matplotlib code intends to create a scatter plot, but it's missing a crucial function call to display the plot. Correct it.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 4, 6]

plt.scatter(x, y)
# Missing plt.show()
```

### Corrected Code:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 4, 6]

plt.scatter(x, y)
plt.title("Simple Scatter Plot") # Added for clarity
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show() # Crucial function call to display the plot
```

**Explanation:** After creating a plot using functions like `plt.plot()`, `plt.scatter()`, `plt.hist()`, etc., you must call `plt.show()` to render and display the plot window. Without `plt.show()`, the plot is created in memory but not displayed to the user.

## Question 100: Conceptual

What is the purpose of `plt.subplots()` in Matplotlib, and when would you use it?

**Answer:** `plt.subplots()` is a Matplotlib function that creates a figure and a set of subplots. It returns a tuple containing a `Figure` object and an `Axes` object (or an array of `Axes` objects if multiple subplots are created).

### Purpose:

- **Convenience:** It's the recommended way to create a figure and one or more subplots in a single call, simplifying the setup for plotting.
- **Structured Layout:** It provides a structured way to arrange multiple plots within a single figure, allowing for comparisons or displaying related data side-by-side.

### When to use it:

- When you need to create a single plot and want direct access to the `Figure` and `Axes` objects for more fine-grained control over the plot.
- When you need to create **multiple plots (subplots)** within the same figure, arranging them in a grid (e.g., 2x2, 1x3).

### Example (Single Subplot):

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots() # Creates a figure and a single subplot (Axes
object)
ax.plot([1, 2, 3], [1, 4, 2])
ax.set_title("Single Subplot")
plt.show()
```

### Example (Multiple Subplots):

```

import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 400)
y_sin = np.sin(x)
y_cos = np.cos(x)

fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8, 6)) # 2 rows, 1
column

axes[0].plot(x, y_sin, label='sin(x)', color='blue')
axes[0].set_title('Sine Wave')
axes[0].legend()

axes[1].plot(x, y_cos, label='cos(x)', color='red')
axes[1].set_title('Cosine Wave')
axes[1].legend()

plt.tight_layout() # Adjusts subplot params for a tight layout
plt.show()

```

## References

[1] Python Documentation: <https://docs.python.org/3/> [2] NumPy Documentation: <https://numpy.org/doc/stable/> [3] Pandas Documentation: <https://pandas.pydata.org/docs/> [4] Matplotlib Documentation: <https://matplotlib.org/stable/> [5] Real Python: <https://realpython.com/> [6] GeeksforGeeks: <https://www.geeksforgeeks.org/>

## Question 56: Coding

Create a 3x3 NumPy array with random integers between 1 and 10. Then, find the sum of each row and the mean of each column.

**Answer:**

```
import numpy as np

# Create a 3x3 array with random integers between 1 and 10
arr = np.random.randint(1, 11, size=(3, 3))
print("Original array:\n", arr)

# Sum of each row (axis=1)
row_sums = np.sum(arr, axis=1)
print("\nSum of each row:", row_sums)

# Mean of each column (axis=0)
col_means = np.mean(arr, axis=0)
print("Mean of each column:", col_means)
```

### Explanation:

- `np.random.randint(1, 11, size=(3, 3))` creates a 3x3 array with random integers from 1 (inclusive) to 11 (exclusive).
- `np.sum(arr, axis=1)` calculates the sum along `axis=1` (rows).
- `np.mean(arr, axis=0)` calculates the mean along `axis=0` (columns).

### Question 57: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.arange(10).reshape(2, 5)
print(arr[0, 2])
print(arr[:, 1])
print(arr[1, :3])
```

### Answer:

```
2
[1 6]
[5 6 7]
```

## Explanation:

- `np.arange(10)` creates an array `[0, 1, ..., 9]`. `reshape(2, 5)` turns it into a 2x5 array.

```
[[0, 1, 2, 3, 4],  
 [5, 6, 7, 8, 9]]
```

- `arr[0, 2]` accesses the element at row 0, column 2, which is `2`.
- `arr[:, 1]` accesses all rows (`:`) and column 1, resulting in `[1, 6]`.
- `arr[1, :3]` accesses row 1 and columns from 0 up to (but not including) 3, resulting in `[5, 6, 7]`.

## Question 58: Debugging/Correction

The following NumPy code intends to stack two 1D arrays vertically, but it uses an incorrect function. Correct it.

```
import numpy as np  
  
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
  
# stacked_arr = np.hstack((arr1, arr2)) # Incorrect for vertical stacking  
# print(stacked_arr)
```

## Corrected Code:

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

stacked_arr = np.vstack((arr1, arr2)) # Correct function for vertical
stacking
print(stacked_arr)
# Output:
# [[1 2 3]
#  [4 5 6]]
```

**Explanation:** `np.hstack()` (horizontal stack) concatenates arrays column-wise. For vertical stacking (row-wise), `np.vstack()` should be used. Both functions take a tuple of arrays as input.

## Question 59: Conceptual

What is the purpose of `np.where()` in NumPy, and how is it used?

**Answer:** `np.where()` is a powerful NumPy function that returns elements chosen from `x` or `y` depending on `condition`. It is essentially a vectorized version of the `if-else` statement, allowing for efficient conditional selection of array elements.

**Usage:** `np.where(condition, x, y)`

- `condition`: A boolean array or array-like object. Where `True`, yield `x`, otherwise yield `y`.
- `x, y`: Arrays or scalars. The elements to choose from.

If only `condition` is provided, `np.where()` returns the indices of elements that are `True`.

**Example:**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Replace values greater than 30 with 0
result = np.where(arr > 30, 0, arr)
print(result) # Output: [10 20 30  0  0]

# Get indices where condition is True
indices = np.where(arr > 30)
print(indices) # Output: (array([3, 4]),)
```

## Question 60: Coding

Given a NumPy array `data = np.array([1, 5, 2, 8, 3, 9, 4])`, use boolean indexing to select all elements greater than 4.

**Answer:**

```
import numpy as np

data = np.array([1, 5, 2, 8, 3, 9, 4])

# Create a boolean array
condition = data > 4
print("Condition array:", condition) # Output: Condition array: [False  True
False  True False  True False]

# Use boolean indexing to select elements
selected_elements = data[condition]
print("Selected elements:", selected_elements) # Output: Selected elements:
[5 8 9]

# Alternatively, in one line:
selected_elements_online = data[data > 4]
print("Selected elements (one line):", selected_elements_online) # Output:
Selected elements (one line): [5 8 9]
```

**Explanation:** Boolean indexing involves creating a boolean array of the same shape as the original array. Where the boolean array is `True`, the corresponding element from

the original array is selected. `data > 4` directly creates this boolean array.

## Question 61: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr + 5)
print(arr * np.array([1, 0, 1, 0, 1]))
```

**Answer:**

```
[ 6  7  8  9 10]
[1 0 3 0 5]
```

**Explanation:**

- `arr + 5` demonstrates broadcasting: the scalar `5` is added to each element of `arr`.
- `arr * np.array([1, 0, 1, 0, 1])` performs element-wise multiplication between the two arrays, as they have compatible shapes.

## Question 62: Debugging/Correction

The following NumPy code intends to calculate the dot product of two matrices, but it uses an incorrect operator for matrix multiplication. Correct it.

```
import numpy as np

matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# dot_product = matrix1 * matrix2 # Incorrect for matrix multiplication
# print(dot_product)
```



## Corrected Code:

```
import numpy as np

matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Correct way to perform matrix multiplication
dot_product = np.dot(matrix1, matrix2) # Using np.dot()
# Or using the @ operator (Python 3.5+)
# dot_product = matrix1 @ matrix2

print(dot_product)
# Output:
# [[19 22]
#  [43 50]]
```

**Explanation:** In NumPy, the `*` operator performs element-wise multiplication (Hadamard product), not matrix multiplication. For matrix multiplication, you should use `np.dot()` or the `@` operator (available in Python 3.5 and later).

## Question 63: Conceptual

What is the difference between `np.array()` and `np.asarray()` ?

**Answer:** Both `np.array()` and `np.asarray()` are used to convert input into a NumPy array, but they have a subtle yet important difference:

- `np.array()` : Always creates a **new array** (a copy) from the input data, even if the input is already a NumPy array of the same `dtype` .
- `np.asarray()` : Creates a new array **only if** the input is not already a NumPy array of the same `dtype` . If the input is already an `ndarray` and its `dtype` matches, it returns a reference to the original array (no copy is made).

### When to use which:

- Use `np.array()` when you explicitly need a **copy** of the data, ensuring that modifications to the new array do not affect the original data source.

- Use `np.asarray()` when you want to ensure the input is an array but want to avoid unnecessary copying if it's already an array. This can be more memory-efficient and faster when dealing with large arrays.

## Question 64: Coding

Create a NumPy array `arr = np.array([10, 20, 30, 40, 50])`. Use NumPy functions to find the maximum value, minimum value, and their respective indices.

**Answer:**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

max_value = np.max(arr)
min_value = np.min(arr)

max_index = np.argmax(arr)
min_index = np.argmin(arr)

print(f"Array: {arr}")
print(f"Maximum value: {max_value} at index: {max_index}") # Output: Maximum
value: 50 at index: 4
print(f"Minimum value: {min_value} at index: {min_index}") # Output: Minimum
value: 10 at index: 0
```

**Explanation:**

- `np.max()` and `np.min()` return the maximum and minimum values in the array, respectively.
- `np.argmax()` and `np.argmin()` return the indices of the first occurrence of the maximum and minimum values, respectively.

## Question 65: Output Prediction

What will be the output of the following NumPy code?

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
reshaped_arr = arr.reshape(2, 3)
print(reshaped_arr)
print(reshaped_arr.flatten())
```

**Answer:**

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

**Explanation:**

- `arr.reshape(2, 3)` changes the shape of the 1D array `arr` into a 2x3 2D array.
- `reshaped_arr.flatten()` returns a copy of the array collapsed into one dimension.

---

## 4. Pandas (25 Questions)

---

### Question 66: Conceptual

What is Pandas, and what are its two primary data structures?

**Answer: Pandas** is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation library for Python. It is built on top of NumPy and provides data structures and operations for manipulating numerical tables and time series.

Its two primary data structures are:

1. **Series:** A one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It is similar to a column in a spreadsheet or a SQL table, or a dictionary with an ordered index.

2. **DataFrame:** A two-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet, SQL table, or a dictionary of Series objects. It is the most commonly used Pandas object.

## Question 67: Coding

Create a Pandas Series from a Python list `[10, 20, 30, 40, 50]` with custom index labels `["a", "b", "c", "d", "e"]`. Then, access the element at index “c”.

**Answer:**

```
import pandas as pd

data = [10, 20, 30, 40, 50]
index_labels = ["a", "b", "c", "d", "e"]

series = pd.Series(data, index=index_labels)
print("Series:\n", series)

# Access element at index "c"
element_c = series["c"]
print("\nElement at index \"c\":", element_c) # Output: Element at index
"c": 30
```

**Explanation:**

- `pd.Series()` is used to create a Series, passing the data and custom index labels.
- Elements can be accessed using their label-based index, similar to dictionary key access.

## Question 68: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
print(df['Age'].mean())
print(df[df['Age'] > 28])
```

**Answer:**

```
30.0
   Name  Age      City
1   Bob   30  Los Angeles
2  Charlie  35    Chicago
```

**Explanation:**

- `df['Age'].mean()` calculates the mean of the 'Age' column, which is  $(25 + 30 + 35) / 3 = 30.0$ .
- `df[df['Age'] > 28]` uses boolean indexing to select rows where the 'Age' is greater than 28. This selects the rows for 'Bob' and 'Charlie'.

## Question 69: Debugging/Correction

The following Pandas code intends to read a CSV file named `data.csv` into a DataFrame, but it might encounter a `FileNotFoundError`. Correct it to handle the case where the file doesn't exist.

```
import pandas as pd

# df = pd.read_csv('data.csv') # This line might cause an error
# print(df.head())
```

**Corrected Code:**

```

import pandas as pd
import os

file_path = 'data.csv'

# Create a dummy CSV file for demonstration if it doesn't exist
if not os.path.exists(file_path):
    dummy_data = {
        'col1': [1, 2, 3],
        'col2': ['A', 'B', 'C']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv(file_path, index=False)
    print(f"Created dummy file: {file_path}")

try:
    df = pd.read_csv(file_path)
    print("DataFrame head:\n", df.head())
except FileNotFoundError:
    print(f"Error: The file {file_path} was not found.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

```

## Explanation:

- The original code would raise a `FileNotFoundError` if `data.csv` did not exist. The corrected code wraps `pd.read_csv()` in a `try-except` block to gracefully handle this specific error.
- For demonstration purposes, a dummy `data.csv` is created if it doesn't exist, so the code can run without manual file creation.

## Question 70: Conceptual

Explain the difference between `.loc` and `.iloc` in Pandas DataFrames.

**Answer:** Both `.loc` and `.iloc` are used for selecting data from Pandas DataFrames, but they differ in how they interpret the indexing arguments:

- `.loc` is primarily **label-based indexing**. It uses the actual row and column labels (names) to select data. It can also take boolean arrays.

- Syntax: `df.loc[row_label, column_label]`
- The end label in slicing is **inclusive**.
- `.iloc` is primarily **integer-location based indexing**. It uses the integer positions (0-based) of rows and columns to select data.
  - Syntax: `df.iloc[row_position, column_position]`
  - The end position in slicing is **exclusive** (like standard Python slicing).

### Example:

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])

print("DataFrame:\n", df)

# Using .loc
print("\n.loc[\"y\", \"B\"]:", df.loc["y", "B"]) # Output: 5
print("\n.loc[\"x\": \"y\", \"A\"]:\n", df.loc["x": "y", "A"]) # Output:
Series with 'x': 1, 'y': 2

# Using .iloc
print("\n.iloc[1, 1]:", df.iloc[1, 1]) # Output: 5
print("\n.iloc[0:2, 0]:\n", df.iloc[0:2, 0]) # Output: Series with 'x': 1,
'y': 2
```

## Question 71: Coding

Given a DataFrame `df` with columns 'A', 'B', 'C', calculate the sum of column 'A', the mean of column 'B', and the maximum value of column 'C'.

### Answer:

```
import pandas as pd

data = {
    'A': [10, 20, 30, 40],
    'B': [1, 2, 3, 4],
    'C': [100, 200, 50, 150]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

sum_A = df['A'].sum()
mean_B = df['B'].mean()
max_C = df['C'].max()

print(f"\nSum of column A: {sum_A}")    # Output: Sum of column A: 100
print(f"Mean of column B: {mean_B}")    # Output: Mean of column B: 2.5
print(f"Max of column C: {max_C}")      # Output: Max of column C: 200
```

**Explanation:** Pandas Series (which columns of a DataFrame are) have built-in methods like `.sum()`, `.mean()`, and `.max()` for common aggregations.

## Question 72: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['A', 'B', 'C']
})
df.set_index('col2', inplace=True)
print(df.loc['B', 'col1'])
```

**Answer:**

2



## Explanation:

- A DataFrame is created.
- `df.set_index('col2', inplace=True)` sets the 'col2' column as the new index of the DataFrame, modifying it in place. The DataFrame now looks like:

```
      col1
col2
A        1
B        2
C        3
```

- `df.loc['B', 'col1']` then uses label-based indexing to select the value in the row labeled 'B' and column 'col1', which is 2.

## Question 73: Debugging/Correction

The following Pandas code intends to drop rows with any missing values, but it uses an incorrect method. Correct it.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8],
    'C': [9, 10, 11, 12]
})

# df_cleaned = df.fillna(0) # Incorrect for dropping rows
# print(df_cleaned)
```

## Corrected Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8],
    'C': [9, 10, 11, 12]
})
print("Original DataFrame:\n", df)

df_cleaned = df.dropna() # Correct method to drop rows with any NaN values
print("\nCleaned DataFrame (dropna):\n", df_cleaned)
# Output:
#    A    B    C
# 0  1.0  5.0   9
# 3  4.0  8.0  12
```

### Explanation:

- `df.fillna(0)` replaces `NaN` values with `0`, it does not remove rows. The original intent was to drop rows with missing values.
- `df.dropna()` is the correct method to remove rows (or columns) containing `NaN` (Not a Number) values. By default, it drops rows if *any* `NaN` is present.

## Question 74: Conceptual

What is the purpose of `groupby()` in Pandas, and provide a simple example.

**Answer:** The `groupby()` method in Pandas is used for grouping rows of a DataFrame together based on some criteria, typically one or more column values. It is a fundamental operation for performing aggregate analysis on subsets of data. The process generally involves three steps:

1. **Splitting:** The data is split into groups based on some criteria.
2. **Applying:** A function (e.g., aggregation, transformation, filtration) is applied to each group independently.
3. **Combining:** The results of these operations are combined into a new DataFrame or Series.

## Example:

```
import pandas as pd

data = {
    'Category': ['A', 'B', 'A', 'B', 'A'],
    'Value': [10, 20, 15, 25, 12]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Group by 'Category' and calculate the sum of 'Value' for each group
grouped_sum = df.groupby('Category')['Value'].sum()
print("\nGrouped Sum:\n", grouped_sum)
# Output:
# Category
# A      37
# B      45
# Name: Value, dtype: int64

# Group by 'Category' and calculate the mean of 'Value' for each group
grouped_mean = df.groupby('Category')['Value'].mean()
print("\nGrouped Mean:\n", grouped_mean)
# Output:
# Category
# A      12.333333
# B      22.500000
# Name: Value, dtype: float64
```

**Explanation:** In this example, the DataFrame is grouped by the ‘Category’ column. For each category (‘A’ and ‘B’), the sum and mean of the ‘Value’ column are calculated, demonstrating how `groupby()` facilitates aggregate analysis.

## Question 75: Coding

Given two DataFrames, `df1` and `df2`, merge them based on a common column ‘ID’.

**Answer:**

```

import pandas as pd

df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'ID': [2, 3, 4],
    'Score': [85, 90, 78]
})

print("DataFrame 1:\n", df1)
print("\nDataFrame 2:\n", df2)

# Merge DataFrames on the 'ID' column (inner join by default)
merged_df = pd.merge(df1, df2, on='ID')
print("\nMerged DataFrame (Inner Join):\n", merged_df)
# Output:
#    ID    Name  Score
# 0    2     Bob    85
# 1    3  Charlie    90

# Example of a left join
left_merged_df = pd.merge(df1, df2, on='ID', how='left')
print("\nMerged DataFrame (Left Join):\n", left_merged_df)
# Output:
#    ID    Name  Score
# 0    1   Alice   NaN
# 1    2     Bob   85.0
# 2    3  Charlie   90.0

```

**Explanation:** `pd.merge()` is used to combine DataFrames based on common columns or indices. By default, it performs an inner join, keeping only the rows where the 'ID' exists in both DataFrames. The `how` parameter can be used to specify other types of joins (e.g., 'left', 'right', 'outer').

## Question 76: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.T)
```

**Answer:**

```
   0  1  2
A  1  2  3
B  4  5  6
```

**Explanation:** `df.T` (or `df.transpose()`) transposes the DataFrame, swapping rows and columns. The original column names ('A', 'B') become the new index, and the original index (0, 1, 2) becomes the new column names.

## Question 77: Debugging/Correction

The following Pandas code intends to apply a function to each element of a Series, but it uses a regular Python loop which is less efficient. Correct it to use a Pandas vectorized operation or `.apply()`.

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])

# result = []
# for x in s:
#     result.append(x * 2)
# new_s = pd.Series(result)
# print(new_s)
```

**Corrected Code (Vectorized Operation):**

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])
print("Original Series:\n", s)

# Vectorized operation (most efficient)
new_s_vectorized = s * 2
print("\nNew Series (vectorized):\n", new_s_vectorized)

# Output:
# 0      2
# 1      4
# 2      6
# 3      8
# 4     10
# dtype: int64
```

### Corrected Code (Using `.apply()` for more complex functions):

```
import pandas as pd

s = pd.Series([1, 2, 3, 4, 5])

def custom_function(x):
    return x * 2 + 1 # More complex operation

new_s_apply = s.apply(custom_function)
print("\nNew Series (apply):\n", new_s_apply)

# Output:
# 0      3
# 1      5
# 2      7
# 3      9
# 4     11
# dtype: int64
```

### Explanation:

- For simple element-wise operations like multiplication, Pandas Series support vectorized operations directly (e.g., `s * 2`), which are highly optimized and much faster than explicit Python loops.

- For more complex, custom functions, the `.apply()` method is the preferred way to apply a function along an axis of a DataFrame or Series. It is still significantly faster than a pure Python loop.

## Question 78: Conceptual

What is a MultiIndex (Hierarchical Index) in Pandas, and when is it useful?

**Answer:** A **MultiIndex**, also known as a **Hierarchical Index**, is a Pandas Index object that allows for multiple levels of indexing on an axis (rows or columns) of a Series or DataFrame. It provides a way to store and work with data that has more than one logical grouping or category.

### When it is useful:

- **Complex Data Structures:** When dealing with data that naturally has multiple levels of categorization, such as time-series data with year and month, or experimental data with different conditions and replicates.
- **Data Aggregation:** It simplifies operations like `groupby()` on multiple levels, allowing for more sophisticated data analysis.
- **Data Selection:** Provides powerful and flexible ways to select subsets of data using multiple index levels.

### Example:

```
import pandas as pd

# Create a MultiIndex DataFrame
index = pd.MultiIndex.from_product([['A', 'B'], [1, 2]], names=['Category',
'Subcategory'])
df = pd.DataFrame({
    'Value': [10, 20, 30, 40],
    'Data': ['x', 'y', 'z', 'w']
}, index=index)
print("MultiIndex DataFrame:\n", df)

# Output:
#
#           Value Data
# Category Subcategory
# A         1      10   x
#           2      20   y
# B         1      30   z
#           2      40   w

# Accessing data with MultiIndex
print("\nAccessing data for Category A:\n", df.loc['A'])

# Output:
#
#           Value Data
# Subcategory
# 1           10   x
# 2           20   y
```

## Question 79: Coding

Create a Pandas DataFrame from a dictionary of lists. Add a new column 'Total' which is the sum of 'Col1' and 'Col2'. Then, drop the 'Col2' column.

**Answer:**



```

import pandas as pd

data = {
    'Col1': [10, 20, 30],
    'Col2': [1, 2, 3],
    'Col3': ['A', 'B', 'C']
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Add 'Total' column
df['Total'] = df['Col1'] + df['Col2']
print("\nDataFrame after adding 'Total':\n", df)

# Drop 'Col2' column
df_modified = df.drop('Col2', axis=1) # axis=1 for columns, inplace=False by default
print("\nDataFrame after dropping 'Col2':\n", df_modified)

# Output:
#      Col1 Col3  Total
# 0      10   A      11
# 1      20   B      22
# 2      30   C      33

```

## Explanation:

- New columns can be added by simply assigning a Series (or a scalar) to a new column name.
- `df.drop()` is used to remove rows or columns. `axis=1` specifies that a column should be dropped. By default, `drop()` returns a new DataFrame; `inplace=True` would modify the original DataFrame.

## Question 80: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': ['P', 'Q', 'P', 'Q']
})

print(df.groupby('B')['A'].sum())
```

**Answer:**

```
B
P    4
Q    6
Name: A, dtype: int64
```

**Explanation:**

- The DataFrame is grouped by the unique values in column 'B' ('P' and 'Q').
- For each group, the sum of the corresponding values in column 'A' is calculated.
  - For 'P':  $1 + 3 = 4$
  - For 'Q':  $2 + 4 = 6$

## Question 81: Debugging/Correction

The following Pandas code intends to rename a column, but it's not correctly applying the change. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'old_name': [1, 2, 3],
    'B': [4, 5, 6]
})

# df.rename(columns={'old_name': 'new_name'}) # Does not modify in-place by default
# print(df.columns)
```

## Corrected Code:

```
import pandas as pd

df = pd.DataFrame({
    'old_name': [1, 2, 3],
    'B': [4, 5, 6]
})
print("Original columns:", df.columns)

# Correct way to rename a column (using inplace=True or reassigning)
df.rename(columns={'old_name': 'new_name'}, inplace=True)
# Alternatively, reassign:
# df = df.rename(columns={'old_name': 'new_name'})

print("New columns:", df.columns) # Output: New columns: Index(['new_name',
'B'], dtype='object')
```

**Explanation:** The `rename()` method, like many Pandas methods, returns a new DataFrame by default and does not modify the original DataFrame in-place unless `inplace=True` is specified. To apply the change, either reassign the result back to the DataFrame (`df = df.rename(...)`) or use `inplace=True`.

## Question 82: Conceptual

What is the purpose of `apply()` and `map()` in Pandas, and when would you use each?

**Answer:** Both `apply()` and `map()` are used to apply functions to Pandas Series or DataFrames, but they serve different purposes and are used in different contexts:

- **`.apply()` :**
  - **Purpose:** Applies a function along an axis of a DataFrame or to the elements of a Series. It is more general and can operate on entire rows/columns or element-wise.
  - **Usage:**
    - On a **Series**: Applies a function to each element.
    - On a **DataFrame**: Applies a function to each row (`axis=1`) or each column (`axis=0`). The function should expect a Series as input and

return a Series or a scalar.

- **When to use:** For more complex operations that cannot be achieved with vectorized NumPy functions, or when you need to operate on entire rows/columns.
- **.map()** :
  - **Purpose:** Element-wise mapping of values in a Series. It is used to substitute each value in a Series with another value, often from a dictionary or another Series.
  - **Usage:** Only works on a **Series**.
  - **When to use:** For substituting values based on a lookup table (dictionary or Series), or when you need to apply a simple element-wise transformation that can be expressed as a mapping.

### Example:

```
import pandas as pd

s = pd.Series([1, 2, 3, 4])

# Using .apply() on a Series
def square(x):
    return x**2
s_apply = s.apply(square)
print("Series with .apply():\n", s_apply)

# Using .map() on a Series with a dictionary
mapping_dict = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
s_map = s.map(mapping_dict)
print("\nSeries with .map():\n", s_map)
```

## Question 83: Coding

Given a DataFrame `df` with a 'Date' column (as strings in 'YYYY-MM-DD' format), convert it to datetime objects and then extract the year and month into new columns 'Year' and 'Month'.

**Answer:**

```

import pandas as pd

df = pd.DataFrame({
    'Date': ['2023-01-15', '2023-02-20', '2024-03-10'],
    'Value': [100, 150, 200]
})

print("Original DataFrame:\n", df)
print("Date column dtype before conversion:", df['Date'].dtype)

# Convert 'Date' column to datetime objects
df['Date'] = pd.to_datetime(df['Date'])
print("\nDate column dtype after conversion:", df['Date'].dtype)

# Extract Year and Month
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month

print("\nDataFrame with Year and Month:\n", df)
# Output:
#           Date  Value  Year  Month
# 0 2023-01-15    100   2023      1
# 1 2023-02-20    150   2023      2
# 2 2024-03-10    200   2024      3

```

### Explanation:

- `pd.to_datetime()` is used to convert a Series of strings into datetime objects. This is crucial for time-series operations.
- Once the column is of `datetime` type, the `.dt` accessor allows easy extraction of date and time components like `year`, `month`, `day`, `hour`, etc.

### Question 84: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.sum(axis=0))
print(df.sum(axis=1))
```

### Answer:

```
A      6
B     15
dtype: int64
0      5
1      7
2      9
dtype: int64
```

### Explanation:

- `df.sum(axis=0)` calculates the sum for each column (default behavior if `axis` is not specified). So, A:  $1+2+3=6$  , B:  $4+5+6=15$  .
- `df.sum(axis=1)` calculates the sum for each row. So, 0:  $1+4=5$  , 1:  $2+5=7$  , 2:  $3+6=9$  .

## Question 85: Debugging/Correction

The following Pandas code intends to fill missing values in a DataFrame with the mean of their respective columns, but it's not correctly handling the `inplace` parameter. Correct it.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8]
})

# df.fillna(df.mean(), inplace=False) # This would return a new DataFrame
# but not assign it
# print(df)
```

## Corrected Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, 7, 8]
})
print("Original DataFrame:\n", df)

# Correct way to fill NaN with column means (using inplace=True or
# reassigning)
df.fillna(df.mean(), inplace=True)
# Alternatively, reassign:
# df = df.fillna(df.mean())

print("\nDataFrame after filling NaNs with column means:\n", df)
# Output:
#      A      B
# 0  1.0  5.0
# 1  2.0  6.666667
# 2  2.333333  7.0
# 3  4.0  8.0
```

**Explanation:** Similar to `rename()`, `fillna()` returns a new DataFrame by default. To modify the original DataFrame, you must either set `inplace=True` or reassign the result of `fillna()` back to the DataFrame variable.

## Question 86: Conceptual

What is a Pivot Table in Pandas, and what is its main use case?

**Answer:** A **Pivot Table** in Pandas is a powerful data summarization tool that allows you to reorganize and aggregate data from a DataFrame. It takes column values from the original DataFrame and transforms them into a new index (rows), columns, and values, often applying an aggregation function (like sum, mean, count) to the data.

**Main Use Case:** Pivot tables are primarily used for **summarizing and analyzing data by one or more categorical variables**. They are excellent for:

- **Multi-dimensional analysis:** Easily view data from different perspectives.
- **Reporting:** Creating summary reports from raw data.
- **Data exploration:** Quickly understand relationships and patterns within your data.

**Example:**

```
import pandas as pd

data = {
    'Region': ['East', 'West', 'East', 'West', 'East'],
    'Product': ['A', 'B', 'A', 'C', 'B'],
    'Sales': [100, 150, 120, 200, 130]
}
df = pd.DataFrame(data)
print("Original DataFrame:\n", df)

# Create a pivot table to show total sales by Region and Product
pivot_table = df.pivot_table(values='Sales', index='Region',
                              columns='Product', aggfunc='sum')
print("\nPivot Table (Total Sales by Region and Product):\n", pivot_table)

# Output:
# Product      A      B      C
# Region
# East      220.0  130.0   NaN
# West       NaN  150.0  200.0
```



## Question 87: Coding

Given a DataFrame `df` with a 'Price' column, create a new column 'Price\_Category' based on the following conditions: 'High' if Price > 100, 'Medium' if Price is between 50 and 100 (inclusive), and 'Low' if Price < 50.

**Answer:**

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Item': ['A', 'B', 'C', 'D', 'E'],
    'Price': [120, 60, 30, 90, 150]
})
print("Original DataFrame:\n", df)

# Define conditions and choices
conditions = [
    (df['Price'] > 100),
    (df['Price'] >= 50) & (df['Price'] <= 100),
    (df['Price'] < 50)
]
choices = ['High', 'Medium', 'Low']

# Use np.select to create the new column
df['Price_Category'] = np.select(conditions, choices, default='Unknown')

print("\nDataFrame with Price_Category:\n", df)

# Output:
#   Item  Price Price_Category
# 0    A    120             High
# 1    B     60           Medium
# 2    C     30             Low
# 3    D     90           Medium
# 4    E    150             High
```

**Explanation:** `np.select()` is a vectorized way to choose values from a list of choices based on a list of conditions. It is more efficient than using multiple `if-elif-else` statements or `.apply()` for this type of conditional logic on DataFrames.

## Question 88: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

s = pd.Series([1, 2, None, 4, 5])
print(s.isnull().sum())
print(s.dropna())
```

**Answer:**

```
1
0    1.0
1    2.0
3    4.0
4    5.0
dtype: float64
```

**Explanation:**

- `s.isnull()` returns a boolean Series indicating where values are `NaN` (or `None`). `s.isnull().sum()` counts the number of `True` values, which is `1` (for the `None` value).
- `s.dropna()` returns a new Series with `NaN` values removed. The original index is preserved for the non-`NaN` values.

## Question 89: Debugging/Correction

The following Pandas code intends to sort a DataFrame by a column in descending order, but it's not correctly specifying the sorting order. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})

# df.sort_values(by='Score') # Sorts in ascending order by default
# print(df)
```

### Corrected Code:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})
print("Original DataFrame:\n", df)

# Correct way to sort in descending order
df_sorted = df.sort_values(by='Score', ascending=False)
print("\nSorted DataFrame (descending by Score):\n", df_sorted)

# Output:
#      Name  Score
# 1    Bob     92
# 0  Alice     85
# 2  Charlie    78
```

**Explanation:** By default, `sort_values()` sorts in ascending order. To sort in descending order, you need to set the `ascending` parameter to `False`.

### Question 90: Conceptual

What is the purpose of `pd.concat()` and `pd.merge()` in Pandas, and when would you use each?

**Answer:** Both `pd.concat()` and `pd.merge()` are used to combine DataFrames, but they do so in different ways and are suitable for different scenarios:

- `pd.concat()` :
  - **Purpose:** Concatenates Pandas objects (Series or DataFrames) along a particular axis (rows or columns). It essentially “stacks” or “joins” them side-by-side or one on top of another.
  - **When to use:** When you want to append DataFrames (add rows) or combine them side-by-side (add columns) without considering common columns/indices for alignment. It’s useful for combining data that has the same structure or when you want to stack data.
- `pd.merge()` :
  - **Purpose:** Combines DataFrames based on common columns or indices, similar to SQL joins. It aligns rows from two DataFrames based on matching values in specified key columns.
  - **When to use:** When you want to combine DataFrames that share one or more common columns (keys) and you need to align the data based on these keys. It’s ideal for relational operations.

**Example (pd.concat()):**

```

import pandas as pd

df1 = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})
df2 = pd.DataFrame({
    'A': [5, 6],
    'B': [7, 8]
})

# Concatenate along rows (default)
concat_rows = pd.concat([df1, df2])
print("\nConcatenated (rows):\n", concat_rows)
# Output:
#    A  B
# 0  1  3
# 1  2  4
# 0  5  7
# 1  6  8

# Concatenate along columns
df3 = pd.DataFrame({
    'C': [9, 10],
    'D': [11, 12]
})
concat_cols = pd.concat([df1, df3], axis=1)
print("\nConcatenated (columns):\n", concat_cols)
# Output:
#    A  B   C   D
# 0  1  3   9  11
# 1  2  4  10  12

```

**Example (pd.merge()):**

```

import pandas as pd

df_left = pd.DataFrame({
    'key': ['K0', 'K1', 'K2', 'K3'],
    'A': ['A0', 'A1', 'A2', 'A3'],
    'B': ['B0', 'B1', 'B2', 'B3']
})

df_right = pd.DataFrame({
    'key': ['K0', 'K1', 'K4', 'K5'],
    'C': ['C0', 'C1', 'C4', 'C5'],
    'D': ['D0', 'D1', 'D4', 'D5']
})

merged_df = pd.merge(df_left, df_right, on='key', how='inner')
print("\nMerged (inner join):\n", merged_df)
# Output:
#   key  A  B  C  D
# 0  K0  A0 B0 C0 D0
# 1  K1  A1 B1 C1 D1

```

## Question 91: Coding

Given a DataFrame `df` with a column 'Values', normalize the values to be between 0 and 1 (Min-Max Normalization).

**Answer:**

```
import pandas as pd

df = pd.DataFrame({
    'Values': [10, 20, 30, 40, 50]
})
print("Original DataFrame:\n", df)

min_val = df['Values'].min()
max_val = df['Values'].max()

df['Normalized_Values'] = (df['Values'] - min_val) / (max_val - min_val)

print("\nDataFrame with Normalized_Values:\n", df)
# Output:
#   Values  Normalized_Values
# 0      10              0.00
# 1      20              0.25
# 2      30              0.50
# 3      40              0.75
# 4      50              1.00
```

**Explanation:** Min-Max Normalization is calculated as  $(X - \min(X)) / (\max(X) - \min(X))$ . Pandas allows for vectorized operations on Series, making this calculation straightforward and efficient.

## Question 92: Output Prediction

What will be the output of the following Pandas code?

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print(df.pipe(lambda x: x + 1))
```

**Answer:**

```
   A  B
0  2  5
1  3  6
2  4  7
```

**Explanation:** The `.pipe()` method allows you to chain together functions that expect a DataFrame or Series as input and return a DataFrame or Series. Here, a lambda function `lambda x: x + 1` is applied to the entire DataFrame, adding 1 to each element.

### Question 93: Debugging/Correction

The following Pandas code intends to reset the index of a DataFrame, but it's not correctly dropping the old index. Correct it.

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])

# df.reset_index() # Keeps the old index as a column by default
# print(df)
```

**Corrected Code:**



```

import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
print("Original DataFrame:\n", df)

# Correct way to reset index and drop the old index
df_reset = df.reset_index(drop=True) # drop=True prevents old index from
becoming a column
print("\nDataFrame after resetting index:\n", df_reset)
# Output:
#      A  B
# 0    1  4
# 1    2  5
# 2    3  6

```

**Explanation:** By default, `reset_index()` converts the old index into a new column. To discard the old index entirely, you need to set the `drop` parameter to `True`.

## Question 94: Conceptual

What is the purpose of `value_counts()` in Pandas Series/DataFrames?

**Answer:** The `value_counts()` method in Pandas is used to return a Series containing counts of unique values. The resulting Series will be in descending order so that the first element is the most frequently occurring element. Excludes NA values by default.

**Main Use Case:** It is extremely useful for **exploratory data analysis (EDA)** to quickly understand the distribution of categorical data or to identify the frequency of different items in a column.

**Example:**

```
import pandas as pd

s = pd.Series(["apple", "banana", "apple", "orange", "banana", "apple"])
print("Original Series:\n", s)

value_counts = s.value_counts()
print("\nValue Counts:\n", value_counts)

# Output:
# apple      3
# banana     2
# orange     1
# dtype: int64
```

## Question 95: Coding

Given a DataFrame `df` with a 'Text' column, convert all text to lowercase and remove leading/trailing whitespace.

**Answer:**

```
import pandas as pd

df = pd.DataFrame({
    'Text': [' Hello World ', ' Python ', 'Pandas']
})
print("Original DataFrame:\n", df)

# Convert to lowercase and strip whitespace
df['Text_Cleaned'] = df['Text'].str.lower().str.strip()

print("\nDataFrame with Text_Cleaned:\n", df)

# Output:
#           Text Text_Cleaned
# 0  Hello World  hello world
# 1      Python      python
# 2      Pandas      pandas
```

**Explanation:** Pandas Series of string type have a special `.str` accessor that provides a rich set of string manipulation methods. `str.lower()` converts to lowercase, and `str.strip()` removes leading/trailing whitespace.

---

## 5. Matplotlib (15 Questions)

---

### Question 96: Conceptual

What is Matplotlib, and what is its primary purpose?

**Answer: Matplotlib** is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is widely used for generating plots, histograms, power spectra, bar charts, error charts, scatterplots, etc.

**Primary Purpose:** Its primary purpose is to **provide a flexible and powerful foundation for creating high-quality data visualizations** in Python. It allows users to produce publication-quality figures in a variety of hardcopy formats and interactive environments across platforms. It is often used for:

- **Exploratory Data Analysis (EDA):** To visualize data distributions, relationships, and patterns.
- **Reporting and Presentation:** To create clear and informative plots for reports, presentations, and academic papers.
- **Customization:** Offers extensive control over every aspect of a figure, from plot styles to text properties.

### Question 97: Coding

Create a simple line plot using Matplotlib with `x = [1, 2, 3, 4]` and `y = [10, 15, 13, 18]`. Add a title "Simple Line Plot" and label the x-axis "X-axis" and y-axis "Y-axis".

**Answer:**

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 15, 13, 18]

plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True) # Optional: Add a grid for better readability
plt.show()
```

### Explanation:

- `plt.plot(x, y)` creates the line plot.
- `plt.title()`, `plt.xlabel()`, and `plt.ylabel()` are used to add a title and axis labels.
- `plt.show()` displays the plot.

### Question 98: Output Prediction

What will be the output (in terms of plot elements) of the following Matplotlib code?

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.figure(figsize=(8, 4))
plt.plot(x, y1, label='sin(x)', color='blue', linestyle='--')
plt.plot(x, y2, label='cos(x)', color='red', linestyle='-')
plt.legend()
plt.show()
```

**Answer:** This code will display a plot with two lines. One line will represent `sin(x)` in blue with a dashed linestyle, and the other will represent `cos(x)` in red with a solid

linestyle. A legend will be displayed, showing the labels 'sin(x)' and 'cos(x)'. The figure will have a width of 8 inches and a height of 4 inches.

### Explanation:

- `plt.figure(figsize=(8, 4))` creates a figure with a specified size.
- Two `plt.plot()` calls are made, each with different `label`, `color`, and `linestyle` arguments.
- `plt.legend()` displays the legend using the labels provided in the `plt.plot()` calls.

### Question 99: Debugging/Correction

The following Matplotlib code intends to create a scatter plot, but it's missing a crucial function call to display the plot. Correct it.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 4, 6]

plt.scatter(x, y)
# Missing plt.show()
```

### Corrected Code:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 4, 6]

plt.scatter(x, y)
plt.title("Simple Scatter Plot") # Added for clarity
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show() # Crucial function call to display the plot
```

**Explanation:** After creating a plot using functions like `plt.plot()`, `plt.scatter()`, `plt.hist()`, etc., you must call `plt.show()` to render and display the plot window. Without `plt.show()`, the plot is created in memory but not displayed to the user.

## Question 100: Conceptual

What is the purpose of `plt.subplots()` in Matplotlib, and when would you use it?

**Answer:** `plt.subplots()` is a Matplotlib function that creates a figure and a set of subplots. It returns a tuple containing a `Figure` object and an `Axes` object (or an array of `Axes` objects if multiple subplots are created).

### Purpose:

- **Convenience:** It's the recommended way to create a figure and one or more subplots in a single call, simplifying the setup for plotting.
- **Structured Layout:** It provides a structured way to arrange multiple plots within a single figure, allowing for comparisons or displaying related data side-by-side.

### When to use it:

- When you need to create a single plot and want direct access to the `Figure` and `Axes` objects for more fine-grained control over the plot.
- When you need to create **multiple plots (subplots)** within the same figure, arranging them in a grid (e.g., 2x2, 1x3).

### Example (Single Subplot):

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots() # Creates a figure and a single subplot (Axes
object)
ax.plot([1, 2, 3], [1, 4, 2])
ax.set_title("Single Subplot")
plt.show()
```

### Example (Multiple Subplots):

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 400)
y_sin = np.sin(x)
y_cos = np.cos(x)

fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8, 6)) # 2 rows, 1
column

axes[0].plot(x, y_sin, label='sin(x)', color='blue')
axes[0].set_title('Sine Wave')
axes[0].legend()

axes[1].plot(x, y_cos, label='cos(x)', color='red')
axes[1].set_title('Cosine Wave')
axes[1].legend()

plt.tight_layout() # Adjusts subplot params for a tight layout
plt.show()
```

---

## References

---

- [1] Python Documentation: <https://docs.python.org/3/> [2] NumPy Documentation: <https://numpy.org/doc/stable/> [3] Pandas Documentation: <https://pandas.pydata.org/docs/> [4] Matplotlib Documentation: <https://matplotlib.org/stable/> [5] Real Python: <https://realpython.com/> [6] GeeksforGeeks: <https://www.geeksforgeeks.org/>